FIAP ON

TRATORES DIGITAIS - HARDWARES PARA IA

CONSTRUINDO O FUTURO COM REDES NEURAIS ARTIFICIAIS

12

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Cérebro humano 7
Figura 2 - Representação simplificada do neurônio biológico 8
Figura 3 - Principais pesquisadores e marcos da história da Inteligência Artificial....9
Figura 4 - Relação entre os conceitos de Inteligência Artificial, Machine Learning e Deep Learning.....12
Figura 5 - Componentes do Perceptron 13
Figura 6 - Principais funções de ativação: Degrau (a) e Sigmoide (b) 14
Figura 7 - Outras funções de ativação 15
Figura 8 - Tabela verdade do E LÓGICO (*and*).....18
Figura 9 - Representação do Perceptron que modelará a tabela verdade do E LÓGICO (*and*) 19
Figura 10 - Tabela verdade e representação visual do AND.....26
Figura 11 - Tabela verdade e representação visual do OR.....26
Figura 12 - Tabela verdade e representação visual do XOR 27
Figura 13 - Representação de uma rede neural artificial multicamada 28
Figura 14 - Imagens representativas do MNIST Dataset 34
Figura 15 - Rede neural para classificação do MNIST Dataset.....35
Figura 16 - Exemplo de equação do segundo grau.....38
Figura 17 - Exemplos de equações complexas.....39
Figura 18 - Até quando treinar redes neurais multicamadas.....41
Figura 19 - Tipos de flores do “Iris Dataset” 45
Figura 20 - Resultado do código-fonte 1 47
Figura 21 - Resultado do código-fonte 3 48
Figura 22 - Resultado do código-fonte 4 49
Figura 23 - Representação da nossa primeira Rede Neural Artificial Multicamadas.50
Figura 24 - Resultado do código-fonte 5 51
Figura 25 - Resultado do código-fonte 6 52
Figura 26 - Resultado do código-fonte 7 54
Figura 27 - Resultado do código-fonte 8 55
Figura 28 - Saída da execução do código-fonte 9.....57
Figura 29 - Saída da execução do código-fonte 11.....59
Figura 30 - Saída da execução do código-fonte 12.....59
Figura 31 - Saída da execução do código-fonte 13.....60
Figura 32 - Saída da execução do código-fonte 14.....61
Figura 33 - Saída da execução do código-fonte 15.....62

TRIAL

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 1 - Resolução da aplicação de um Perceptron simples.....16
Quadro 2 - Análise do valor do imóvel pelo Perceptron 17
Quadro 3 - Predição para x1=x2=0 19
Quadro 4 - Processo de atualização dos pesos do Perceptron.....21
Quadro 5 - Processo de atualização dos pesos do Perceptron.....22
Quadro 6 - Cálculo do número de pesos e bias da rede neural da figura “Rede neural para classificação do MNIST Dataset” 36

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE TABELAS

Tabela 1 - Perceptron modelado para análise de valor de imóveis.....17
Tabela 2 - Modelagem do Perceptron considerando W1=W2=0.....20
Tabela 3 - Modelagem do Perceptron considerando W1=W2=0,1.....21
Tabela 4 - Modelagem do Perceptron considerando W1=W2=0,2.....22
Tabela 5 - Modelagem do Perceptron considerando W1=W2=0,5.....22

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Imports iniciais e leitura da base.....46
Código-fonte 2 - Divisão dos dados em treino e teste.....47
Código-fonte 3 - Definição e treinamento do Perceptron 48
Código-fonte 4 - Plotando o Iris Dataset 49
Código-fonte 5 - Primeiro modelo com Rede Neural Multicamadas.....51
Código-fonte 6 - Segundo modelo com Rede Neural Multicamadas.....52
Código-fonte 7 - Terceiro modelo com Rede Neural Multicamadas.....53
Código-fonte 8 - Quarto modelo com Rede Neural Multicamadas 54
Código-fonte 9 - Leitura do dataset advertising.csv 57
Código-fonte 10 - Separação de dados da base “advertising.csv” 58
Código-fonte 11 - Gerando o modelo baseline (regressão linear).....58
Código-fonte 12 - Gerando o modelo RNA preliminar.....59
Código-fonte 13 - Gerando o modelo RNA secundário – Seed 97.....60
Código-fonte 14 - Gerando o modelo RNA secundário – Seed 93.....61
Código-fonte 15 - Gerando o modelo RNA secundário – Seed 3.....61

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 CONSTRUINDO O FUTURO COM REDES NEURAIS ARTIFICIAIS 7
1.1 Introdução 7
1.1.1 Uma Inspiração Biológica 7
1.1.2 História da IA e sua relação com Redes Neurais 8
1.1.3 Aplicações de Redes Neurais 10

2 PERCEPTRON 13
2.1 Definição e componentes 13
2.1.1 Funções de ativação 14
2.1.2 Exemplo de uso do Perceptron 16
2.2 Treinamento do Perceptron 18
2.2.1 Intuição do processo 18
2.2.2 Consolidação das etapas de treinamento do Perceptron 23
2.2.3 Considerações sobre a taxa de aprendizagem 25

3 REDES NEURAIS MULTICAMADAS 26
3.1 Por que precisamos de redes multicamadas? 26
3.2 Componentes de redes multicamadas 28
3.3 Arquitetura 30
3.3.1 Escolha da topologia 30
3.3.2 Escolha da quantidade de camadas 31
3.3.3 Escolha da quantidade de neurônios 32
3.4 Treinamento da rede 33
3.4.1 Introdução e estratégias de treinamento 33
3.4.2 Backpropagation 37
3.4.3 Até quando treinar redes neurais multicamadas? 41
3.5 Hiperparâmetros 42

4 HANDS ON: MODELAGENS PREDITIVAS COM REDES NEURAIS 45
4.1 Sobre a base de dados 45
4.2 Modelagem via Perceptron 45
4.3 Modelagem via Redes Multicamadas 50
4.4 Redes Multicamadas de Regressão 56
4.4.1 Primeiro modelo com Rede Neural Artificial 59
4.4.2 Aprimorando o modelo de Rede Neural Artificial 60

REFERÊNCIAS 63

GLOSSÁRIO 65

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 CONSTRUINDO O FUTURO COM REDES NEURAIS ARTIFICIAIS

1.1 Introdução

1.1.1 Uma Inspiração Biológica

O aprendizado é o motor da evolução humana, um processo intrincado que nos capacita a adquirir conhecimentos e a nos adaptar a um ambiente em constante transformação. Mas como ocorre esse aprendizado? A resposta está nas complexas redes de conexões presentes no nosso cérebro.

O cérebro humano é uma estrutura fascinante, formada por bilhões de neurônios interligados a milhares de outros. Essas conexões são fundamentais para o aprendizado. Ao absorvermos novos conhecimentos, as sinapses — as junções entre os neurônios — se fortalecem. Esse fenômeno, conhecido como plasticidade sináptica, é o que sustenta nossa habilidade de desenvolver novas competências e assimilar informações.

Figura 1 - Cérebro humano
Fonte: Portal do médico (2021)

Lembra das aulas de Biologia? As redes neurais biológicas serviram de inspiração para a criação das Redes Neurais Artificiais (RNAs). Formadas por neurônios interconectados, essas redes permitem que cada neurônio receba e transmita sinais elétricos e químicos. O modo como as redes biológicas processam

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

informações, criam associações e se adaptam ao ambiente foi o modelo que guiou o desenvolvimento das redes neurais artificiais.

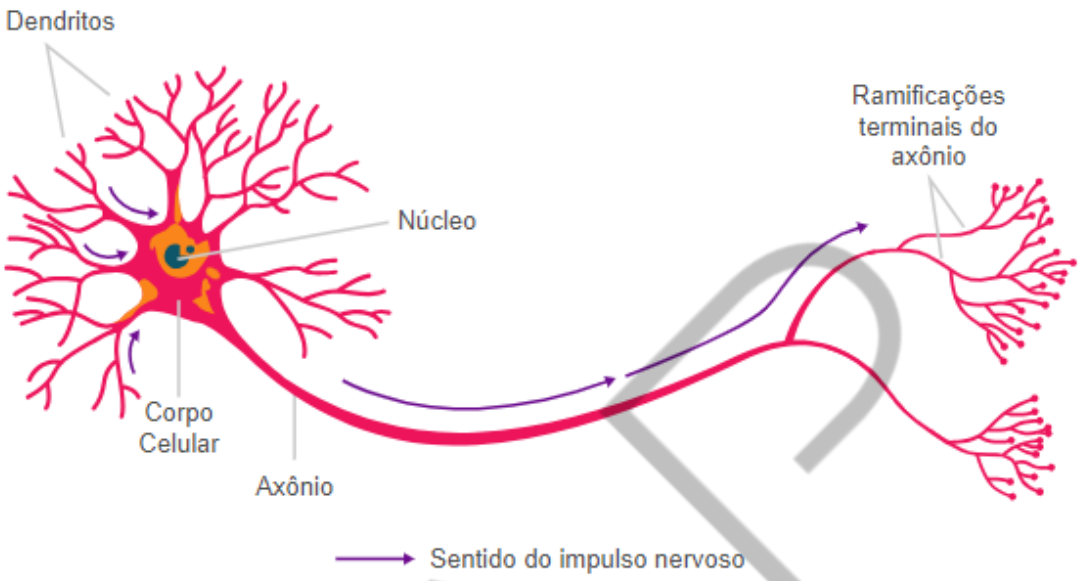


Figura 2 - Representação simplificada do neurônio biológico
Fonte: Deep Learning Book (2024), adaptado por FIAP (2025)

1.1.2 História da IA e sua relação com Redes Neurais

A história da Inteligência Artificial (IA) remonta à década de 1950, quando cientistas começaram a explorar maneiras de **criar máquinas que pudessem simular processos cognitivos humanos**, como resolução de problemas e tomada de decisões. A pesquisa em IA nasceu da interseção entre matemática, lógica, cibernética e neurociência, com a ambição de criar sistemas que pudessem “pensar”.

No entanto, o conceito de RNA é ainda mais antigo, com suas origens nos estudos de neurofisiologia realizados na década de 1940. **Warren McCulloch** e **Walter Pitts** propuseram o primeiro modelo matemático de um **neurônio artificial** em 1943, demonstrando como redes de neurônios simples poderiam realizar cálculos lógicos básicos, como ilustrado na figura “Principais pesquisadores e marcos da história da Inteligência Artificial”. Esse modelo rudimentar estabeleceu as bases para a conexão entre IA e Redes Neurais.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

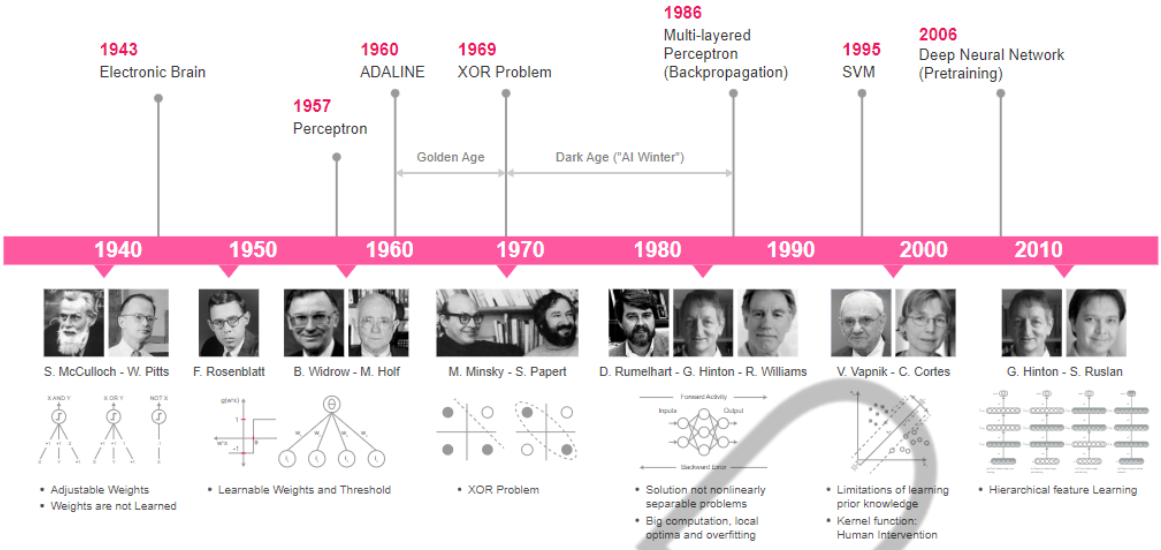


Figura 3 - Principais pesquisadores e marcos da história da Inteligência Artificial
Fonte: Medium (2024), adaptado por FIAP (2025)

Nos anos 1950, **Frank Rosenblatt apresentou o Perceptron**, a primeira implementação prática de uma rede neural. Esse modelo simples tinha a capacidade de aprender a classificar dados por meio de um algoritmo de ajuste de pesos. Embora promissor, o Perceptron era limitado e incapaz de resolver problemas complexos, como os de lógica não linear. Essas restrições, combinadas com o entusiasmo crescente em outros métodos de IA, levaram ao que ficou conhecido como o "**Inverno da IA**", um período de estagnação na pesquisa.

Foi apenas na década de 1980 que as Redes Neurais ganharam novo fôlego com o desenvolvimento do **algoritmo de backpropagation**, uma técnica eficiente para ajustar os pesos em redes multicamadas, tornando possível resolver problemas mais complexos. Nessa mesma época, a convergência de avanços em hardware computacional e a disponibilidade de grandes conjuntos de dados começaram a abrir novas possibilidades para as redes neurais.

Na década de 2010, o surgimento do **Deep Learning** marcou um ponto de virada. Redes neurais profundas, compostas por muitas camadas, passaram a oferecer um desempenho sem precedentes em tarefas como reconhecimento de imagens, tradução de idiomas e jogos complexos. A combinação de arquiteturas avançadas, como as redes convolucionais e recorrentes, com o aumento exponencial da capacidade computacional, consolidou as redes neurais como um dos pilares da IA moderna.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.1.3 Aplicações de Redes Neurais

As RNAs têm revolucionado diversas áreas ao permitir que sistemas computacionais aprendam e executem tarefas complexas, frequentemente superando as capacidades humanas em determinadas aplicações. Inspiradas no funcionamento do cérebro humano, as RNAs são utilizadas para processar, interpretar e analisar grandes volumes de dados, oferecendo soluções inovadoras em diferentes setores. Algumas das principais áreas de aplicação incluem:

- **Visão Computacional:** as redes neurais, especialmente as Redes Neurais Convolucionais (CNNs), são amplamente usadas para análise de imagens e vídeos. Elas desempenham um papel crucial em:
  - Reconhecimento facial: utilizado em segurança, autenticação biométrica e sistemas de monitoramento.
  - Detecção de objetos: identificação de veículos em trânsito, análise de radiografias médicas e monitoramento de ambientes.
  - Análise de imagens médicas: auxílio no diagnóstico de doenças, como a detecção de tumores ou análise de exames como tomografias e ressonâncias.
- **Processamento de Linguagem Natural (PLN):** as redes neurais profundas impulsionam muitas aplicações que lidam com texto e linguagem:
  - Tradução automática: modelos como o Google Translate utilizam redes neurais para traduzir idiomas de forma precisa.
  - Chatbots e assistentes virtuais: sistemas como Alexa, Siri e Google Assistant são baseados em RNA para entender e responder a comandos.
  - Análise de sentimentos: determinação do tom emocional em postagens de mídia social, avaliações de produtos ou feedback de clientes.
- **Financeiro e Negócios:** no setor financeiro, as RNAs são aplicadas para:
  - Detecção de fraudes: identificação de transações anômalas em tempo real.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

  - Previsão de mercado: as Redes Recorrentes (RNNs) ajudam a analisar séries temporais para prever tendências em ações e mercados financeiros.
  - Gerenciamento de riscos: modelos preditivos baseados em RNA avaliam a probabilidade de inadimplência e outros riscos.
- **Automação e Robótica:** as redes neurais desempenham um papel central em robótica e sistemas autônomos:
  - Veículos autônomos: RNAs processam dados de sensores, câmeras e radares para permitir a navegação segura.
  - Robótica industrial: redes treinadas otimizam a eficiência de robôs em linhas de montagem.
  - Exploração espacial: RNAs ajudam robôs a operar em ambientes desconhecidos, como em missões em Marte.
- **Sustentabilidade e Meio Ambiente:** as RNAs estão sendo utilizadas para enfrentar desafios ambientais:
  - Previsão do clima: redes ajudam na modelagem de padrões meteorológicos e previsão de desastres naturais.
  - Gestão de energia: otimização do consumo de energia em redes elétricas inteligentes.
  - Monitoramento ambiental: análise de dados de satélite para rastrear desmatamento, mudanças climáticas e biodiversidade.

Estas e outras aplicações de redes neurais continuam a crescer com os avanços em hardware, algoritmos e disponibilidade de dados. Seja para resolver problemas complexos, automatizar processos ou criar experiências mais personalizadas, as RNA estão moldando o futuro em praticamente todos os campos da ciência, tecnologia e negócios.

Nesta disciplina, exploraremos como redes neurais artificiais são projetadas, treinadas e evoluem para Deep Learning. Embora as redes neurais artificiais sejam classificadas como algoritmos bio-inspirados, elas também fazem parte do grupo de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

algoritmos de aprendizado de máquina. Na verdade, o Deep Learning, que consiste em redes neurais profundas, é a técnica mais promissora em aplicações como Visão Computacional e Processamento de Linguagem Natural atualmente. Percebe como tudo está interligado?

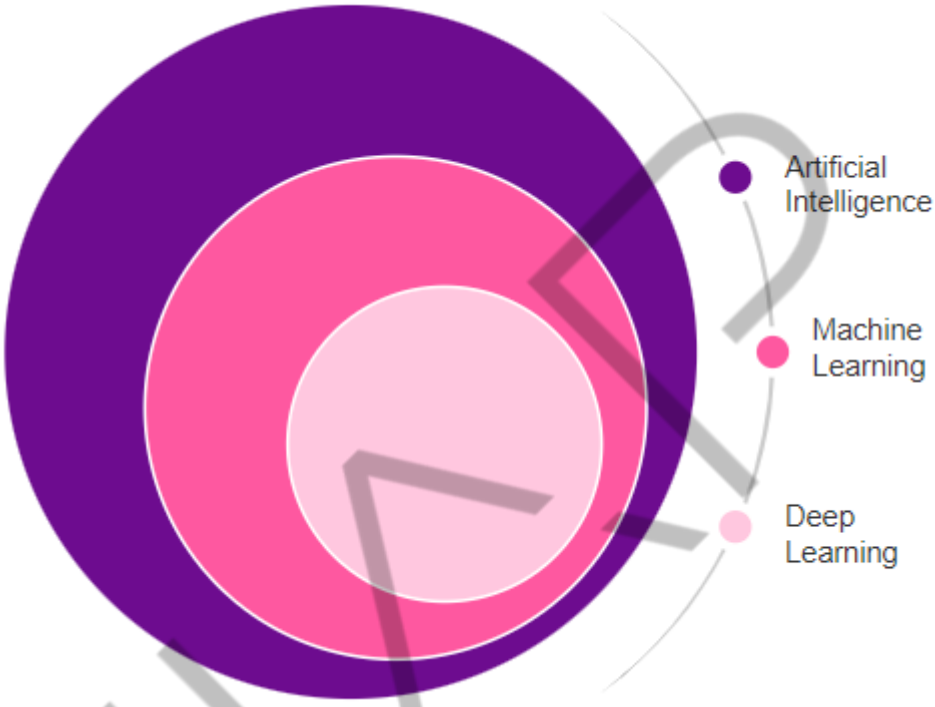


Figura 4 - Relação entre os conceitos de Inteligência Artificial, Machine Learning e Deep Learning
Fonte: Advancing Analytic (2021), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 PERCEPTRON

2.1 Definição e componentes

O Perceptron é um dos blocos fundamentais no mundo das Redes Neurais Artificiais. Embora seja um modelo simples, sua compreensão é crucial para construir redes neurais mais complexas que lidam com tarefas sofisticadas de reconhecimento de padrões, processamento de linguagem natural e muito mais.

Para entendê-lo, vamos começar pela sua definição. O Perceptron é um modelo simples de neurônio artificial, inspirado pelo funcionamento dos neurônios biológicos. Ele foi desenvolvido por Frank Rosenblatt na década de 1950. Os componentes principais do Perceptron estão ilustrados na figura “Componentes do Perceptron”.

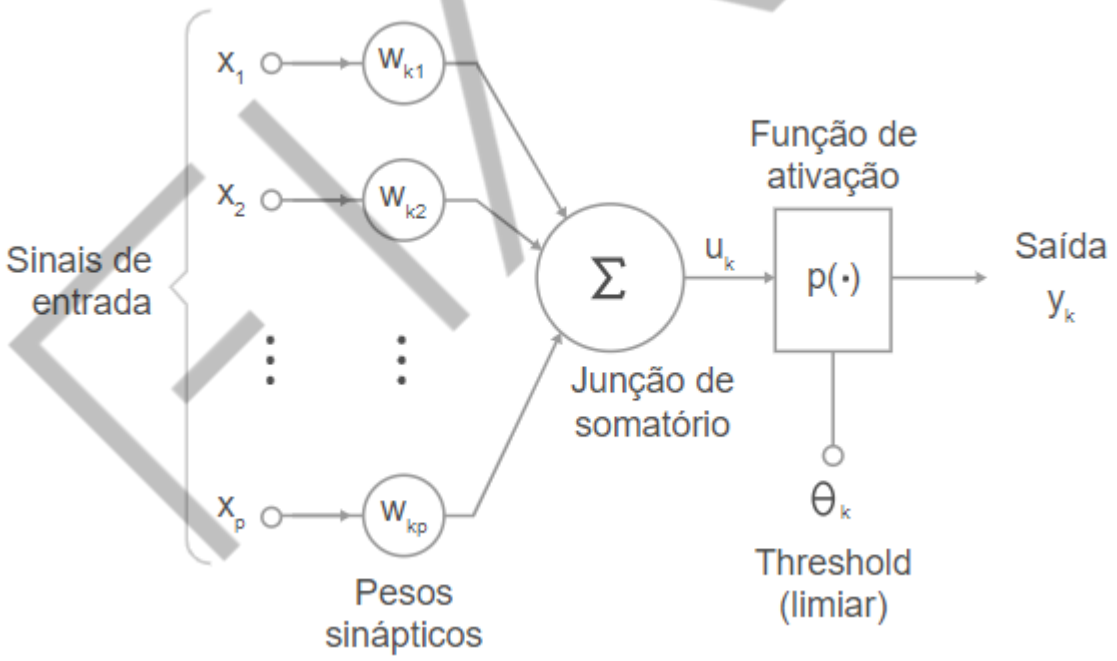


Figura 5 - Componentes do Perceptron
Fonte: Deep Learning Book (2024), adaptado por FIAP (2025)

Em suma, o Perceptron é um algoritmo de **classificação binária** que toma entradas, aplica pesos, soma essas entradas, e em seguida, passa o resultado por uma função de ativação. Essa função de ativação decide se o Perceptron deve

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

“disparar” (produzir uma saída de 1) ou não (produzir uma saída de 0). Matematicamente, a saída do Perceptron é calculada como a soma ponderada das entradas mais um viés — do inglês “*bias*” — (um valor adicional usado para ajustar o limite de ativação).

Devido a sua simplicidade, o Perceptron possui aplicações práticas limitadas às questões de “sim ou não”. Por exemplo, pode ser treinado para distinguir entre números escritos à mão que representam dígitos 0 e 1.

2.1.1 Funções de ativação

A resposta final do Perceptron é definida pela **função de ativação**. As funções de ativação desempenham um papel crucial no funcionamento do Perceptron. Elas determinam se o neurônio deve ser ativado (produzir uma saída diferente de zero) ou não. Duas das funções de ativação mais comuns são a função degrau (*step*) e a função sigmoide. Observe a figura “Principais funções de ativação: Degrau (a) e Sigmoide (b)”.

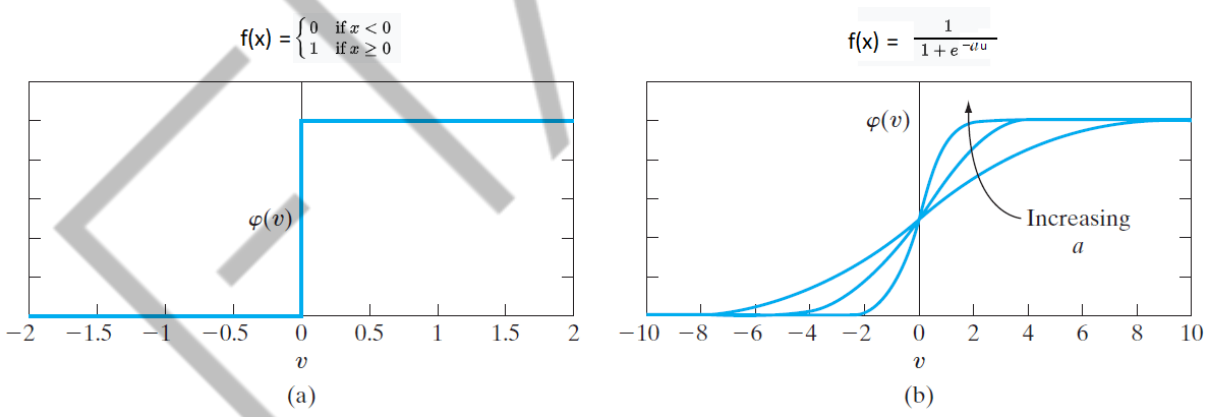


Figura 6 - Principais funções de ativação: Degrau (a) e Sigmoide (b)
Fonte: Researchgate (2018)

A **função degrau** é simples: se a soma ponderada das entradas exceder um certo limite, a saída é 1; caso contrário, é 0. A **função sigmoide**, por sua vez, produz saídas contínuas entre 0 e 1, o que é útil para problemas de probabilidade.

Na prática há uma ampla gama de funções de ativação, cada uma com sua curva característica e função de geração. A figura “Outras funções de ativação” ilustra algumas delas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

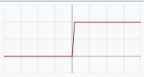
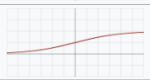
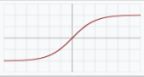
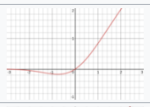
Name	Plot	Function, $g(x)$	Derivative of g, $g'(x)$	Range	Order of continuity
Identity		x	1	$(-\infty, \infty)$	C^∞
Binary step		$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	0	$\{0, 1\}$	C^{-1}
Logistic, sigmoid, or soft step		$\sigma(x) \doteq \frac{1}{1+e^{-x}}$	$g(x)(1-g(x))$	$(0, 1)$	C^∞
Hyperbolic tangent (tanh)		$\tanh(x) \doteq \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$1 - g(x)^2$	$(-1, 1)$	C^∞
Soboleva modified hyperbolic tangent (smht)		$\operatorname{smht}(x) \doteq \frac{e^{ax} - e^{-bx}}{e^{cx} + e^{-dx}}$		$(-1, 1)$	C^∞
Rectified linear unit (ReLU)[8]		$(x)^+ \doteq \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases} = \max(0, x) = x\mathbf{1}_{x>0}$	$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \end{cases}$	$[0, \infty)$	C^0
Gaussian Error Linear Unit (GELU)[2]		$\frac{1}{2}x\left(1 + \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right)\right) = x\Phi(x)$	$\Phi(x) + x\phi(x)$	$(-0.17\ldots, \infty)$	C^∞
Softplus[9]		$\ln(1 + e^x)$	$\frac{1}{1+e^{-x}}$	$(0, \infty)$	C^∞

Figura 7 - Outras funções de ativação
Fonte: Wikipédia (2024)

Podemos detalhar outras além das funções “degrau” e “sigmoide”:

- **Função linear (ou identidade — *identity*):** é uma das funções de ativação mais simples e diretas usadas em redes neurais. O principal objetivo da função linear é realizar uma operação de identidade, ou seja, ela simplesmente passa os valores de entrada sem modificação para a camada seguinte. Ou seja, a saída da função linear é igual à entrada.

- **ReLU (*Rectified Linear Unit*):** a função ReLU é uma das mais populares em redes profundas. Ela é definida como f(x) = max(0, x), isso significa que retorna zero para entradas negativas e a própria entrada para valores positivos. O ReLU ajuda a superar o problema de gradientes desaparecidos (veremos o significado disso adiante), acelerando o treinamento e sendo muito eficiente em termos computacionais.

- **Leaky ReLU:** o Leaky ReLU é uma variante do ReLU que permite um pequeno valor para entradas negativas (por exemplo, f(x) = 0.01x quando x < 0). Isso ajuda a mitigar o problema de neurônios "mortos" (onde a saída sempre é zero) em algumas unidades do ReLU.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Função Tangente Hiperbólica (Tanh):** a função tanh produz saídas no intervalo entre -1 e 1. Ela é frequentemente usada em redes neurais, pois é uma alternativa à função sigmoide e pode ajudar a mitigar o problema de gradientes desaparecidos. Vale mencionar que Tanh é utilizada principalmente em camadas ocultas.

A importância de ter várias opções de funções de ativação reside na flexibilidade que elas oferecem. **Diferentes problemas podem se beneficiar de diferentes funções de ativação, dependendo das características dos dados e da arquitetura da rede**. Além disso, em redes profundas, a escolha da função de ativação pode afetar o sucesso do treinamento, evitando problemas como gradientes desaparecidos e explosão de gradientes (conheceremos tais problemas e aprenderemos como mitigá-los adiante).

2.1.2 Exemplo de uso do Perceptron

Vamos para um **exercício teórico simples**. Dado um Perceptron cujas entradas (x1, x2 e x3) são respectivamente 1, 2 e 3, e os pesos (w1, w2, w3) são respectivamente 0.1, 0.2 e 0.3, calcule a saída desse Perceptron considerando uma função de ativação degrau (limite=0) e bias=0. A resolução é:

Resolução do exercício teórico

```
X = [1, 2, 3]        #Entradas
W = [0.1, 0.2, 0.3] #Pesos
b = 0                #Viés

Se u = W . X + b, então:
u = x1*w1 + x2*w2 + x3*w3 + b
  = 1*0.1 + 2*0.2 + 3*0.3 + 0
  = 0.1 + 0.4 + 0.9 + 0
u = 1.4

Se vamos usar a função degrau, então:
     f(u) = 1, se u≥0; 0, se u<0

Como u = 1.4 ≥ 0, então:

f(u) = 1

Portanto, este Perceptron está ativado!
```

Quadro 1 - Resolução da aplicação de um Perceptron simples
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Agora, vamos colocar o mesmo exercício sob uma perspectiva prática. Imagine que nosso Perceptron deve responder à seguinte questão: dadas as características de um apartamento, seu preço de venda está acima do mercado? Para isso, as características serão apresentadas na tabela a seguir:

Variável	Peso	Valor	Descrição
x1	0,3	600.000	Preço (R$)
x2	0,05	130	Metragem (m²)
x3	0,1	4	Número de quartos (unidades)
Limite	N/A	180.000	Limiar para ajuste decisão da função degrau
Bias	N/A	-5000	Valor do viés a ser aplicado

Tabela 1 - Perceptron modelado para análise de valor de imóveis
Fonte: Elaborado pelo autor (2024)

A resolução está no quadro:

Resolução do exercício de avaliação de imóvel

```
X = [x1, x2, x3]      #Entradas
X = [600000, 130, 4] #Entradas
W = [0.3, 0.05, 0.1] #Pesos
b = -5000            #Viés

Se u = W . X + b, então:
u = x1*w1 + x2*w2 + x3*w3 + b
  = 600000*0.3 + 130*0.05 + 4*0.1 + (-5000)
  = 180000 + 6,5 + 0,4 - 5000
u = 175006,9

Se vamos usar a função degrau, com limite=180000 então:
     f(u) = 1, se u≥180000; 0, se u<0

Como u = 175006,9 < 180000, então:

f(u) = 0

Portanto, este Perceptron não está ativado! Logo, o imóvel não
está acima do valor de mercado.
```

Quadro 2 - Análise do valor do imóvel pelo Perceptron
Fonte: Elaborado pelo autor (2024)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.2 Treinamento do Perceptron

Acabamos de ver duas aplicações do Perceptron, utilizando-o para obter respostas de perguntas relacionadas a classificações binárias. Isso só foi possível porque o Perceptron já estava “configurado”, com os pesos, e bias informados.

O que aconteceria nos exemplos anteriores caso os pesos e bias não estivessem disponíveis? Nós não conseguiríamos utilizá-los.

A aplicação que simulamos reflete o emprego de um modelo preditivo já desenvolvido para a tomada de decisão. Caso os pesos não fossem dados, teríamos de encontrá-los automaticamente, por meio da análise de exemplos. Pois, **encontrar os pesos e bias é justamente treinar o Perceptron**.

2.2.1 Intuição do processo

Para exemplificar o processo de treinamento do Perceptron, vamos modelar um problema simples: o “E LÓGICO” (operação *AND* em condicionais). Sabemos que a tabela verdade do “E LÓGICO” é dada pela figura a seguir:

A	B	A e B
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
FALSE	FALSE	FALSE

Figura 8 - Tabela verdade do E LÓGICO (*and*)
Fonte: Treina Web (2020)

O Perceptron que modelará este problema pode ser representado pela figura “Representação do Perceptron que modelará a tabela verdade do E LÓGICO (*and*)”, tendo duas entradas (x1, x2), e dois pesos a serem descobertos (W1, W2). Para simplificar, consideraremos bias=0 e a função de ativação será: *f(u)* ➔ *f(u)=1 se u=1; do contrário f(u)=0.*

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

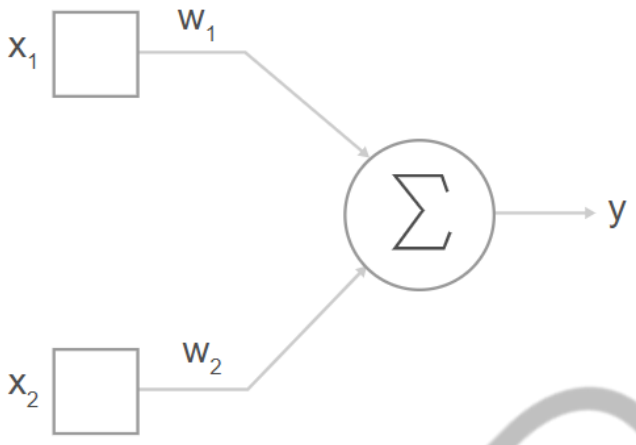


Figura 9 - Representação do Perceptron que modelará a tabela verdade do E LÓGICO (*and*)
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2025)

Como já mencionado, treinar o Perceptron é descobrir os valores de pesos e bias que melhor modelam o problema apresentado como um conjunto de exemplos (ou seja, reduz o erro total das classificações). No nosso caso, sem ocorrer a perda de generalidade, desconsideraremos o bias (bias=0), e focaremos apenas nos pesos.

Nossos exemplos são as quatro linhas da tabela verdade do "E LÓGICO". Precisamos então inicializar os pesos com um valor qualquer. **Vamos começar com W1 = W2 = 0.** Será que este conjunto de pesos modela adequadamente o problema? Vejamos no quadro o processo de "predição" quando x1=x2=0:

Treinamento do Perceptron – Predição para x1=x2=0

```
X = [x1, x2]    #Entradas
X = [0, 0]      #Entradas
W = [0.0, 0.0]  #Pesos
b = 0           #Viés

Se u = W . X + b, então:
u = x1*w1 + x2*w2 + b
  = 0*0.0 + 0*0.0 + 0
  = 0 + 0 + 0
u = 0

Considerando: f(u) → f(u)=1 se u=1; do contrário f(u)=0

Como u = 0, então:

f(u) = 0

Portanto, este Perceptron não está ativado e a saída predita é 0.
```

Quadro 3 - Predição para x1=x2=0
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vamos observar na tabela “Modelagem do Perceptron considerando W1=W2=0” o resultado completo da reprodução dos passos ilustrados no quadro “Predição para x1=x2=0”, mantendo os valores dos pesos, mas alterando as entradas conforme as linhas da tabela verdade:

x1	x2	Saída esperada	f(u)	Saída predita	Status
0	0	0	f(0)	0	Acertou
0	1	0	f(0)	0	Acertou
1	0	0	f(0)	0	Acertou
1	1	1	f(0)	0	Errou

Tabela 2 - Modelagem do Perceptron considerando W1=W2=0
Fonte: Elaborado pelo autor (2024)

A tabela “Modelagem do Perceptron considerando W1=W2=0” mostra que o conjunto de pesos W1=W2=0 não modelam o “E LÓGICO”, pois funcionou apenas para 3 das 4 linhas. Ou seja, ainda há erro na classificação.

Assim, temos que calcular os **erros da predição e atualizar os pesos**. O **cálculo dos erros** é realizado pela soma do erro de cada exemplo, calculado a partir da diferença entre a saída esperada e a saída predita. No ponto acima, o erro total da predição foi 1 (0+0+0+1).

A **atualização dos pesos** segue a fórmula dada pela seguinte equação:

$$Wnovo = Wantigo + (taxa\ de\ aprendizado \times erro \times entrada)$$

A fórmula mostra que atualizar os pesos é um processo iterativo, sendo que o novo valor do peso a ser explorado depende do valor corrente do peso e de um <u>delta</u>, cujo valor é a multiplicação do valor da entrada associado a este peso, o erro da predição e a **taxa de aprendizado**.

A **taxa de aprendizado**, também conhecida como “*learning rate*” em inglês, é um hiperparâmetro crucial em algoritmos de aprendizado de máquina, incluindo redes neurais, que determina o tamanho dos passos que um modelo dá durante o treinamento ao ajustar seus pesos e bias com base no erro calculado. A taxa de aprendizado é um valor escalar positivo que controla a rapidez com que o modelo se adapta aos dados de treinamento. Discutiremos mais sobre a taxa de aprendizado

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

adiante. Por enquanto, considere como um valor estabelecido antes de iniciarmos o treinamento do Perceptron (ou da Rede Neural).

Ok, vamos focar na última linha da tabela “Modelagem do Perceptron considerando W1=W2=0”. Quando aplicamos W1=W2=0 nos valores x1=x2=1, há um erro na predição. Portanto, tal erro indica a necessidade de atualizarmos os pesos. Vamos considerar uma taxa de aprendizado igual a 0,1. A atualização segue os passos do quadro a seguir:

Treinamento do Perceptron – Ajuste dos pesos para x1=x2=1

```
erro = SaídaEsperada - SaídaPredita
erro = 1 - 0
erro = 1

W1novo = W1antigo + (taxa de aprendizado × erro × entrada1)
W1novo = 0 + (0,1 × 1 × 1)
W1novo = 0 + 0,1
W1novo = 0,1

W2novo = W2antigo + (taxa de aprendizado × erro × entrada2)
W2novo = 0 + (0,1 × 1 × 1)
W2novo = 0 + 0,1
W2novo = 0,1

Portanto, na nova iteração, recalcularemos as saídas preditas e o
status da Tabela 2 utilizando W1=W2=0,1.
```

Quadro 4 - Processo de atualização dos pesos do Perceptron
Fonte: Elaborado pelo autor (2024)

Ao utilizarmos W1=W2=0,1 no cálculo das predições de cada linha da tabela verdade do “E LÓGICO”, chegaríamos nos valores da tabela a seguir:

x1	x2	Saída esperada	f(u)	Saída predita	Status
0	0	0	f(0)	0	Acertou
0	1	0	f(0,1)	0	Acertou
1	0	0	f(0,1)	0	Acertou
1	1	1	f(0,2)	0	Errou

Tabela 3 - Modelagem do Perceptron considerando W1=W2=0,1
Fonte: Elaborado pelo autor (2024)

Mesmo com os pesos atualizados continuamos tendo erro na classificação, justamente no caso x1=x2=1. Portanto, devemos atualizar os pesos novamente, seguindo o protocolo previamente introduzido. Ao atualizar os valores do quadro “Processo de atualização dos pesos do Perceptron”, chegaríamos no processo ilustrado no quadro a seguir:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Treinamento do Perceptron – Ajuste dos pesos para x1=x2=1

```
erro = SaídaEsperada - SaídaPredita
erro = 1 - 0
erro = 1

W1novo = W1antigo + (taxa de aprendizado × erro × entrada1)
W1novo = 0,1 + (0,1 × 1 × 1)
W1novo = 0,1 + 0,1
W1novo = 0,2

W2novo = W2antigo + (taxa de aprendizado × erro × entrada2)
W2novo = 0,1 + (0,1 × 1 × 1)
W2novo = 0,1 + 0,1
W2novo = 0,2

Portanto, na nova iteração, recalcularemos as saídas preditas e o
status da Tabela 2 utilizando W1=W2=0,2.
```

Quadro 5 - Processo de atualização dos pesos do Perceptron
Fonte: Elaborado pelo autor (2024)

Ao utilizarmos W1=W2=0,2 no cálculo das predições de cada linha da tabela verdade do “E LÓGICO”, chegaríamos nos valores da tabela a seguir:

x1	x2	Saída esperada	f(u)	Saída predita	Status
0	0	0	f(0)	0	Acertou
0	1	0	f(0,2)	0	Acertou
1	0	0	f(0,2)	0	Acertou
1	1	1	f(0,4)	0	Errou

Tabela 4 - Modelagem do Perceptron considerando W1=W2=0,2
Fonte: Elaborado pelo autor (2024)

Ainda assim haveria erro de classificação! Neste ponto você já deve ter entendido o processo. Portanto, antes de continuar, siga com os cálculos à mão e encontre os pesos ideais para este Perceptron.

... (estou aguardando seus cálculos!)...

Certo! Após seus cálculos, você deve ter chegado no valor de W1=W2=0,5, que produz os resultados da tabela “Modelagem do Perceptron considerando W1=W2=0,5”:

x1	x2	Saída esperada	f(u)	Saída predita	Status
0	0	0	f(0)	0	Acertou
0	1	0	f(0,5)	0	Acertou
1	0	0	f(0,5)	0	Acertou
1	1	1	f(1)	1	Acertou

Tabela 5 - Modelagem do Perceptron considerando W1=W2=0,5
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Pronto! Agora podemos falar que nosso Perceptron está TREINADO, pois encontramos os pesos que modelam o problema apresentado, sem erros.

Importante: precisamos fazer algumas considerações sobre o exemplo apresentado! Leia os pontos a seguir com atenção!

Considerações sobre o treinamento do Perceptron que modela o "E LÓGICO":

- Nosso problema possuía apenas quatro exemplos (afinal, a tabela verdade do AND para duas variáveis possui apenas quatro combinações de valores). Em aplicações reais de Machine Learning não faz o menor sentido termos um conjunto de treinamento tão pequeno assim!
- No nosso exemplo, nós não avaliamos o modelo final no conjunto de TESTE porque ele não existe. Em aplicações reais, com datasets reais, isso é fundamental.
- Nós iniciamos os pesos com valor 0 (zero). Na prática podemos iniciar os pesos com valores aleatórios.
- Nosso modelo parou o treinamento quando o erro da "rede neural" foi igual a zero (todos os exemplos do treino foram corretamente classificados). Na prática isso é inviável: o TREINO seria dividido em TREINO e VALIDAÇÃO, e certamente haveria um percentual de erro na VALIDAÇÃO. **O ponto central do treinamento não é treinar até o erro ser zero, mas até não conseguir melhorar mais os acertos** (ou seja, chegar num erro mínimo do problema, não necessariamente zero).
- O processo de atualização levava a valores iguais para W1 e W2. Isso foi uma coincidência para o exemplo didático apresentado. Na prática, cada peso pode ter valores distintos.

2.2.2 Consolidação das etapas de treinamento do Perceptron

Acabamos de passar pelo processo intuitivo de treinamento do Perceptron. Você deve ter notado que o processo segue uma série de etapas bem definidas. Portanto, é natural pensarmos neste processo como um algoritmo. Os passos básicos do treinamento do Perceptron são:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1. **Inicialização dos pesos:** inicialmente, os pesos das entradas são atribuídos a valores aleatórios ou pequenos valores próximos a zero.
2. **Alimentação dos dados:** um exemplo de treinamento é apresentado ao Perceptron. Os valores das entradas são multiplicados pelos pesos correspondentes, e a soma ponderada é calculada.
3. **Aplicação da função de ativação:** a soma ponderada é então passada por uma função de ativação, geralmente uma função degrau. Esta função determina se o Perceptron deve ativar (gerar uma saída de 1) ou não ativar (gerar uma saída de 0) com base na soma ponderada.
4. **Cálculo do erro:** o resultado da função de ativação é comparado com o rótulo real do exemplo de treinamento. O erro é calculado subtraindo a saída real da saída prevista pelo Perceptron.
5. **Ajuste dos pesos:** os pesos das entradas são atualizados com base no erro calculado e na taxa de aprendizado (*learning rate*). A fórmula de atualização é: $Wnovo = Wantigo + (taxa\ de\ aprendizado \times erro \times entrada)$. Os pesos são ajustados na direção que reduzirá o erro na próxima iteração.
6. **Iteração:** os passos 2 a 5 são repetidos para todos os exemplos de treinamento até que um critério de parada seja atingido. Isso pode ser:
    a. Quando existe um número fixo de iterações.
    b. Quando o erro atinge um valor mínimo.
    c. Quando o Perceptron converge e classifica os exemplos corretamente.
7. **Convergência:** o treinamento do Perceptron converge quando não é mais capaz de cometer erros em um conjunto específico de exemplos de treinamento. Isso significa que ele aprendeu a dividir corretamente os exemplos em suas classes.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.2.3 Considerações sobre a taxa de aprendizagem

A escolha apropriada da taxa de aprendizado é crítica para o sucesso do treinamento de um modelo. Vejamos algumas considerações importantes:

- **Escolha da taxa de aprendizado:** a escolha da taxa de aprendizado depende do problema específico e muitas vezes requer ajuste e experimentação. Valores típicos estão na faixa de 0,1 a 0,0001.
- **Taxa de aprendizado pequena:** se a taxa de aprendizado for muito pequena, o treinamento pode ser extremamente lento, pois os passos nos ajustes dos pesos são minúsculos. Além disso, pode ser fácil ficar preso em mínimos locais na função de perda (que mede o erro total).
- **Taxa de aprendizado grande:** se a taxa de aprendizado for muito grande, o treinamento pode ser instável e divergir, fazendo com que os pesos oscilem e nunca alcancem uma solução ideal.
- **Técnicas de ajuste:** para lidar com o problema de escolher uma taxa de aprendizado adequada, as técnicas de ajuste automático da taxa de aprendizado, como o "*learning rate decay*", e o uso de otimizadores adaptativos, como o Adam, são comumente empregadas. Veremos estas técnicas adiante.
- **Monitoramento e ajuste:** durante o treinamento, é importante monitorar o desempenho do modelo no conjunto de validação e ajustar a taxa de aprendizado, se necessário, para otimizar a convergência e evitar problemas como o overfitting.

Em resumo, a taxa de aprendizado é um hiperparâmetro crítico que influencia a eficiência e a estabilidade do treinamento de modelos de aprendizado de máquina, incluindo redes neurais. Encontrar o valor adequado para a taxa de aprendizado é, muitas vezes, uma parte essencial do processo de treinamento de um modelo.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 REDES NEURAIS MULTICAMADAS

3.1 Por que precisamos de redes multicamadas?

Em nossa exploração do universo das redes neurais, encontramos o Perceptron, um elemento central nessa jornada. Ele é particularmente eficaz para resolver problemas lineares, como a separação de pontos em um plano por meio de uma linha reta.

Imagine, por exemplo, que você precisa prever se um cliente realizará uma compra com base em seu histórico de transações. Se o histórico puder ser representado por um único valor, como o total gasto em compras anteriores, o Perceptron pode criar uma linha de decisão simples que separa os clientes propensos a comprar daqueles que não compram.

Como exemplo, utilizamos o Perceptron para modelar o “E LÓGICO” (AND), cuja tabela verdade e representação gráfica estão mostradas na seguinte figura:

A	B	A e B
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
FALSE	FALSE	FALSE

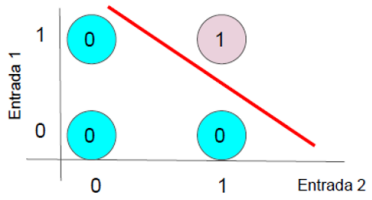


Figura 10 - Tabela verdade e representação visual do AND
Fonte: Portal do Médico (2021)

Na parte direita da figura “Tabela verdade e representação visual do AND”, podemos ver que o “E LÓGICO” pode ser modelado linearmente, com a classificação representada pela linha vermelha. De forma semelhante, o “OU LÓGICO” (OR) também pode ser linearmente modelado, como mostrado na figura a seguir:

A	B	A ou B
TRUE	TRUE	TRUE
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE
FALSE	FALSE	FALSE

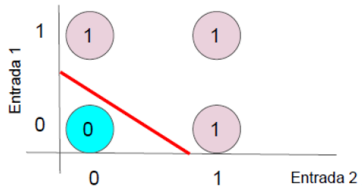


Figura 11 - Tabela verdade e representação visual do OR
Fonte: Portal do Médico (2021)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O lado direito da figura “Tabela verdade e representação visual do OR” mostra que o “OU LÓGICO” pode ser representado como um problema linear, com a separação feita pela linha em vermelho.

Entretanto, no mundo real, nos deparamos com desafios muito mais complexos e não lineares. Imagine, por exemplo, a tarefa de classificar imagens de gatos e cachorros. Cada imagem é composta por milhares de pixels, e traçar uma simples linha reta para separá-las seria inviável. Esse tipo de problema exige uma abordagem mais avançada, e é aí que entram as redes neurais multicamadas.

Para ilustrar, consideremos o “XOR” (ou “EXCLUSIVO”), cuja lógica só é verdadeira quando apenas uma das condições é satisfeita. A tabela verdade e a representação visual do “XOR” estão na figura a seguir:

A	B	A xor B
TRUE	TRUE	FALSE
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE
FALSE	FALSE	FALSE

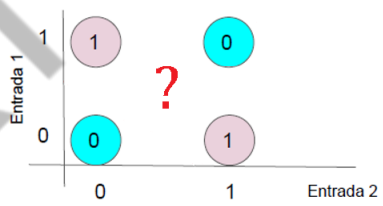


Figura 12 - Tabela verdade e representação visual do XOR
Fonte: Portal do Médico (2021)

Na figura “Tabela verdade e representação visual do XOR”, observamos que não é possível traçar uma linha reta que separe adequadamente as classes ativadas (1) das não ativadas (0). Esse problema exige representações mais sofisticadas e complexas.

A solução para esses desafios está na introdução de mais camadas nas redes neurais, criando as chamadas “redes multicamadas”. Essas redes são compostas por várias camadas de neurônios interconectados, permitindo a modelagem de representações mais abstratas e complexas dos dados. No caso da classificação de imagens de gatos e cachorros, as redes multicamadas podem aprender automaticamente a identificar padrões como bordas, texturas e formas, o que torna a classificação muito mais precisa.

Portanto, as redes multicamadas são essenciais para resolver problemas complexos e não lineares do mundo real. Elas são a base de modelos poderosos e

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

flexíveis que podem enfrentar uma ampla gama de aplicações, desde reconhecimento de fala e visão computacional até tradução automática e outras tarefas avançadas.

3.2 Componentes de redes multicamadas

As redes neurais multicamadas são formadas por diversos componentes que colaboram para aprender e modelar relações complexas nos dados. Sua representação básica é apresentada na figura a seguir:

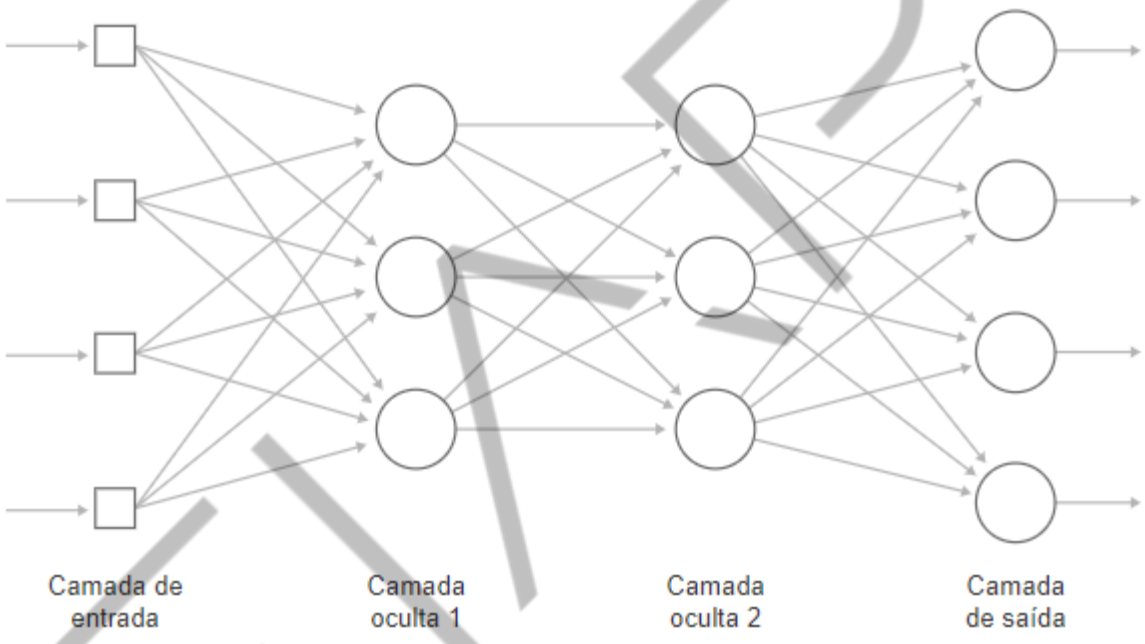


Figura 13 - Representação de uma rede neural artificial multicamada
Fonte: GeeksforGeeks (2024), adaptado por FIAP (2025)

Vamos examinar esses componentes essenciais. Primeiramente, as camadas da rede, que se dividem em três tipos principais:

- **Camada de Entrada (*Input Layer*):** é a primeira camada da rede, responsável por receber os dados de entrada. Cada neurônio nessa camada representa uma característica (feature) específica dos dados fornecidos.
- **Camadas Ocultas (*Hidden Layers*):** situadas entre a camada de entrada e a camada de saída, essas camadas intermediárias não são diretamente acessíveis. É nelas que ocorre a transformação e processamento dos dados. A figura “Representação de uma rede neural artificial multicamada”

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

ilustra duas camadas ocultas, mas uma rede pode ter mais ou menos camadas ocultas, conforme a complexidade necessária.

- **Camada de Saída (*Output Layer*):** última camada da rede, encarregada de produzir as saídas ou previsões desejadas. O número de neurônios nessa camada depende do tipo de problema a ser resolvido (classificação, regressão etc). Utilizando o exemplo da figura “Representação de uma rede neural artificial multicamada”, podemos notar que há quatro neurônios na camada de saída, indicando um problema de classificação com quatro classes distintas.

Observa-se que os neurônios estão interconectados. Cada conexão entre neurônios possui um peso associado, motivo pelo qual são chamadas de “conexões ponderadas” (*weighted connections*). Esses pesos determinam a importância das informações transmitidas de um neurônio para outro durante o aprendizado.

Além do que é mostrado na figura “Representação de uma rede neural artificial multicamada”, existem funções fundamentais nas arquiteturas de redes neurais multicamadas:

- **Funções de Ativação (*Activation Functions*):** cada neurônio, exceto os da camada de entrada, aplica uma função de ativação aos seus *inputs* ponderados. Essas funções introduzem não linearidades na rede, permitindo que ela represente relações complexas nos dados.
- **Função de Perda (*Loss Function*):** utilizada para avaliar o desempenho da rede em relação às saídas desejadas. Durante o treinamento, o objetivo é minimizar essa função, ajustando os pesos e os vieses (*biases*).

Também é importante mencionar que, para treinar a rede neural, é necessário um algoritmo de otimização que ajuste os pesos e vieses com base na função de perda. Além disso, conceitos como regularização (para evitar overfitting) e ajustes de hiperparâmetros (como a taxa de aprendizado) são essenciais para otimizar o treinamento. Esses tópicos mais avançados e extremamente importantes serão detalhados ainda neste capítulo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Esses são os componentes principais de uma rede neural multicamada. Cada elemento desempenha um papel vital no processo de aprendizado e na capacidade da rede de modelar relações complexas nos dados.

3.3 Arquitetura

Acabamos de conhecer os componentes típicos de uma rede neural multicamadas. Agora, **aprenderemos a configurar a arquitetura das redes neurais multicamadas**. A arquitetura da rede é definida por um conjunto de escolhas que são feitas antes do início do treinamento. Uma vez que a arquitetura da rede está montada, ela não é alterada em tempo de treinamento.

Para exemplificar a diferença, observe que o treinamento da rede é descobrir o valor adequado de cada peso. Portanto, o valor do peso se altera em tempo de treinamento. Mas, se definirmos que a arquitetura da rede em questão terá quatro camadas ocultas, tal característica permanecerá imutável ao longo do treinamento.

Definir a arquitetura da rede neural refere-se ao processo de especificar sua estrutura, ou seja, o **número de camadas**, o **número de neurônios** em cada camada e a **forma como essas camadas estão interconectadas (topologia)**. A arquitetura é um dos principais fatores que determinam a capacidade e a capacidade de generalização de uma rede neural.

3.3.1 Escolha da topologia

A topologia se refere à organização geral das camadas. A mais comum é rede **completamente conectada** (*fully connected*), representada na figura “Representação de uma rede neural artificial multicamada”. Nesta topologia, cada neurônio ou unidade em uma camada está conectado a todos os neurônios na camada seguinte. Isso significa que todas as saídas de uma camada são utilizadas como entradas para cada neurônio na camada seguinte, criando uma rede densa de conexões.

A escolha da topologia correta depende do problema que você está tentando resolver e dos dados disponíveis. É uma parte fundamental do projeto de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

uma rede neural e geralmente envolve experimentação e ajustes para encontrar a configuração que melhor se adapta ao seu problema específico.

Além das redes completamente conectadas, existem arquiteturas como redes convolucionais (CNNs) para processamento de imagens e redes recorrentes (RNNs) para sequências temporais. Por enquanto, focaremos nas redes feedforward completamente conectadas.

Importante: uma rede feedforward, também conhecida como rede neural artificial de alimentação direta, é o tipo mais simples e comum de rede neural. Nesse tipo de rede, **as informações fluem apenas em uma direção, da camada de entrada para a camada de saída, sem formar ciclos**.

3.3.2 Escolha da quantidade de camadas

As redes neurais geralmente consistem em uma camada de entrada (*input*), uma ou mais camadas ocultas (*hidden layers*) e uma camada de saída (*output*).

A escolha do número de camadas ocultas em uma rede neural é um aspecto crítico do projeto e treinamento da rede, pois afeta diretamente sua capacidade de modelar relacionamentos complexos nos dados.

De modo geral, considere os seguintes aspectos:

- **Natureza do problema:** a escolha do número de camadas ocultas deve ser orientada pelo problema que você está tentando resolver. Problemas simples podem ser tratados com uma única camada oculta, enquanto problemas mais complexos podem exigir múltiplas camadas.
- **Complexidade dos dados:** se os dados contiverem padrões não lineares complexos, as camadas ocultas adicionais podem ajudar a capturar esses padrões. Se os dados forem relativamente simples e lineares, camadas adicionais podem ser desnecessárias.
- **Experimentação:** a experimentação desempenha um papel fundamental na determinação do número de camadas ocultas. Você pode começar com uma arquitetura simples e, se a rede não estiver aprendendo bem ou estiver subajustando (underfitting) os dados, considere adicionar camadas ocultas para aumentar a capacidade da rede.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Empirismo:** o aprendizado de máquina muitas vezes envolve uma dose de empirismo. Experimente diferentes arquiteturas, desde redes com uma única camada oculta até redes com várias camadas ocultas, e avalie o desempenho em um conjunto de validação para determinar qual arquitetura funciona melhor para o seu problema.

Em resumo, a escolha do número de camadas ocultas é uma tarefa que envolve experimentação e ajustes. Não há uma regra rígida que se aplique a todos os cenários, e a seleção ideal depende da natureza do problema e dos dados. Portanto, é importante estar disposto a iterar e otimizar a arquitetura da rede com base no desempenho observado.

3.3.3 Escolha da quantidade de neurônios

A escolha da quantidade de neurônios em cada camada também é um fator debatível. Não há uma regra 100% assertiva nesta escolha, pois ela também depende do problema sendo modelado e dos dados empregados no treinamento da rede.

Jeff Heaton, no seu livro "*Introduction to Neural Networks for Java*", sugere três abordagens para escolha da quantidade de neurônios em uma camada oculta. Para efeitos de exemplificação, considere uma rede com 30 neurônios na camada de entrada e 2 neurônios na camada de saída. As abordagens são:

1. **O número de neurônios ocultos deve estar entre o tamanho da camada de entrada e o da camada de saída**. No nosso exemplo, o valor de (30+2)/2 = 16 neurônios.
2. **O número de neurônios escondidos deve ser 2/3 do tamanho da camada de entrada, mais o tamanho da camada de saída**. Assim, a camada escondida no nosso exemplo deve conter (30*2/3)+2 = 22 neurônios.
3. **O número de neurônios escondidos deve ser menor que o dobro do tamanho da camada de entrada**. Ou seja, no nosso exemplo, a camada escondida deve conter menos que 60 neurônios.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Qualquer uma destas abordagens é um bom “chute” para criar sua arquitetura inicial. Porém, **e se tivermos mais de uma camada oculta?** Para mais de uma camada oculta basta seguir o raciocínio usando a primeira camada oculta como entrada para a próxima.

Lembre-se de que a escolha da quantidade de neurônios não é uma regra fixa e pode variar de problema para problema. É importante iterar e experimentar diferentes arquiteturas para encontrar aquela que melhor funciona para o seu caso.

3.4 Treinamento da rede

3.4.1 Introdução e estratégias de treinamento

Vimos que problemas complexos exigem redes mais complexas. Para nos guiar nos conceitos de treinamento de redes neurais multicamadas, imagine que tenhamos imagens de dígitos escritos à mão e queremos reconhecer automaticamente que dígitos são estes.

O problema de reconhecer dígitos manuscritos é uma tarefa de classificação de padrões em visão computacional e aprendizado de máquina. Onde, trata-se de desenvolver um modelo capaz de identificar e classificar corretamente os dígitos escritos à mão (geralmente de 0 a 9) em imagens digitais. Cada imagem contém um único dígito manuscrito que pode variar em estilo, tamanho, inclinação e posicionamento dentro da imagem. A figura “Imagens representativas do MNIST Dataset” ilustra parte do MNIST, um dataset popular construído para treinar algoritmos nesta tarefa.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 14 - Imagens representativas do MNIST Dataset
Fonte: Wikipédia (2024)

O objetivo é desenvolver um sistema que possa “ler” e identificar os dígitos escritos à mão com alta precisão, independentemente da caligrafia do autor. Esse problema é relevante em diversas aplicações do mundo real, como em sistemas de reconhecimento de caracteres em documentos digitalizados, na classificação automática de códigos postais em correspondências, no reconhecimento de números de telefone em formulários manuscritos, entre outros.

Claramente não podemos tratar o problema com o Perceptron, exigindo uma rede neural mais sofisticada. Vamos utilizar a rede **parcialmente** representada na figura “Rede neural para classificação do MNIST Dataset”, indicando uma rede feedforward, totalmente conectada, contendo quatro camadas: a entrada com 784 neurônios; a 1ª camada oculta com 16 neurônios; a 2ª camada oculta com 32 neurônios; e a camada de decisão com 10 neurônios. Não podemos representar a rede completa pois a imagem seria um caos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

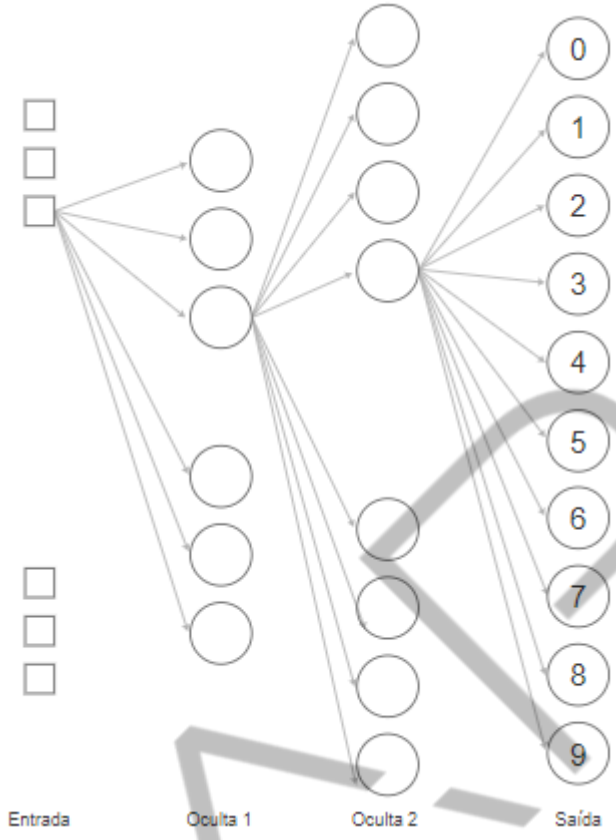


Figura 15 - Rede neural para classificação do MNIST Dataset
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2025)

Por que esta representação? As imagens do MNIST possuem 28x28 pixels, assim, cada imagem contém 784 pixels. A camada de entrada contém um neurônio para cada pixel da entrada. A camada de saída obrigatoriamente precisa ter 10 neurônios, pois temos um problema de classificação com 10 classes (cada um dos dígitos entre 0 e 9). Escolhemos duas camadas ocultas de forma arbitrária, e a quantidade de neurônios em cada uma delas seguiu a heurística proposta por Jeff Heaton.

Quantos pesos e bias são necessários na arquitetura proposta? Tente calcular! A solução está no quadro a seguir:

Cálculo do número de pesos e bias da rede ilustrada na figura "Rede neural para classificação do MNIST Dataset"
Para calcular o número total de pesos e bias necessários para essa rede neural, precisamos levar em consideração as conexões entre as camadas: • Camada de Entrada → 1ª Camada Oculta: ○ Pesos: 784 (entradas) x 16 (neurônios) = 12.544 pesos

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Bias: 16 (neurônios) = 16 bias

- 1ª Camada Oculta → 2ª Camada Oculta:
  - Pesos: 16 (neurônios da camada anterior) x 32 (neurônios da camada atual) = 512 pesos
  - Bias: 32 (neurônios da camada atual) = 32 bias
- 2ª Camada Oculta → Camada de Saída:
  - Pesos: 32 (neurônios da camada anterior) x 10 (neurônios de saída) = 320 pesos
  - Bias: 10 (neurônios de saída) = 10 bias

Portanto, o número total de pesos é 12.544 (da primeira camada) + 512 (da segunda camada) + 320 (da camada de saída) = **13.376 pesos**, e o número total de bias é 16 (da primeira camada) + 32 (da segunda camada) + 10 (da camada de saída) = **58 bias**.

Quadro 6 - Cálculo do número de pesos e bias da rede neural da figura "Rede neural para classificação do MNIST Dataset"
Fonte: Elaborado pelo autor (2024)

Como encontrar os pesos e bias do problema apresentado? Podemos usar a **força bruta**, tentando combinações diferentes para valores dos pesos e bias, até encontrar aquela que mais minimizaria o erro de classificação. Entretanto, utilizar esta estratégia levaria muito, mas muito tempo, para convergir e não é nada prática.

Aprendemos que é possível atualizar os pesos de acordo com uma fórmula que depende dos valores correntes, mas se o peso inicial está "ruim" e tentarmos "corrigir" na direção oposta à esperada, os novos valores estarão cada vez mais distantes do que deveriam ser. Precisamos de uma estratégia mais robusta!

A estratégia robusta a ser empregada é denominada **Backpropagation** (retropropagação). O Backpropagation é um algoritmo fundamental no treinamento de redes neurais artificiais. Ele é utilizado para ajustar os pesos das conexões entre neurônios de forma a minimizar o erro entre as saídas previstas pela rede e os valores reais desejados. O processo envolve a retropropagação do erro da camada de saída para as camadas ocultas, ajustando gradualmente os pesos em direção a um mínimo global na função de erro.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Comparando com a força bruta, o Backpropagation é um método muito mais eficiente e inteligente para treinar redes neurais. Enquanto a força bruta envolveria tentativas exaustivas de ajustar os pesos de forma aleatória e verificar os resultados, o Backpropagation utiliza informações precisas sobre o erro da rede em relação aos dados de treinamento. Isso permite que a rede aprenda de maneira mais rápida e eficaz, evitando ajustes aleatórios e demorados.

3.4.2 Backpropagation

O Backpropagation é o motor por trás do treinamento de redes neurais artificiais multicamadas. Sua intuição é ajustar os pesos das conexões entre neurônios de forma a minimizar o erro entre as saídas previstas pela rede e os valores reais desejados. A pergunta que nos direciona é:

Como atualizar os pesos da rede de forma eficiente (rápida), sem prejudicar as métricas de desempenho?

Antes de mais nada, devemos lembrar das principais funções relacionadas com as redes neurais:

- **Função de ativação:** retorna se um neurônio será ativado ou não.
- **Função de perda:** calcula o erro daquela predição (resposta correta – resposta calculada).
- **Função de custo:** erro médio de aprendizagem (média dos resultados da função de perda).

Repare que a “função de custo” pode ser reescrita utilizando apenas pesos, bias e funções de ativação, pois ela é a média dos erros, que depende do valor calculado baseado na função de ativação (a qual depende da soma ponderada das *entradas*, dos *pesos*, e do *bias*).

Portanto, “treinar a rede neural” passa a ser encarado como um **problema de otimização**: como podemos otimizar a função de custo? Felizmente, existem meios matemáticos para isso!

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vamos começar com o conceito de “**vetor gradiente**”. O gradiente é uma ferramenta matemática que indica a direção na qual obtém-se o **crescimento máximo** de uma função qualquer. Considerando que y = f(x), como podemos variar “x” de modo a incrementar “y”? Observe a figura:

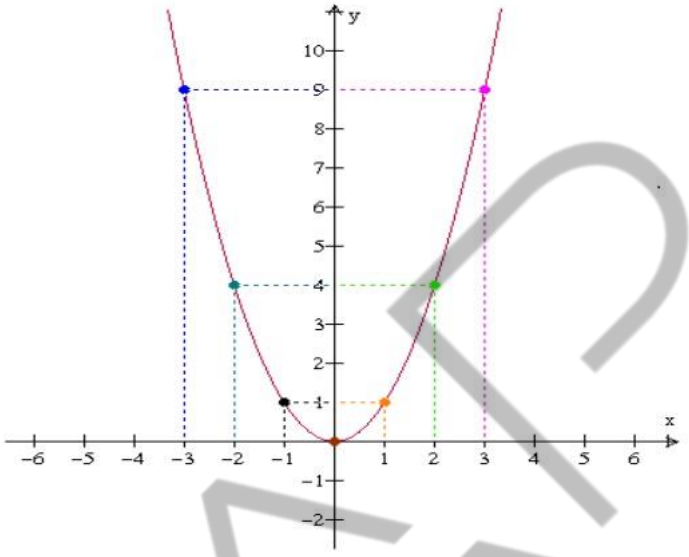


Figura 16 - Exemplo de equação do segundo grau
Fonte: Dever de Casa Web Blogspot (2017)

De acordo com a função ilustrada na figura “Exemplo de equação do segundo grau”, para maximizar esta função (aumentar “y”), podemos **aumentar o valor de “x”** — pois f(x=3) > f(x=2) — ou até mesmo **diminuir o valor de “x”** — pois f(x=-3) > f(x=-2).

Esse caso foi fácil, basta olhar para a função que o “sentido” da maximização está posto. No entanto, em funções mais complexas encontrar tal “sentido” pode não ser tão óbvio, como ilustrado na figura “Exemplos de equações complexas”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

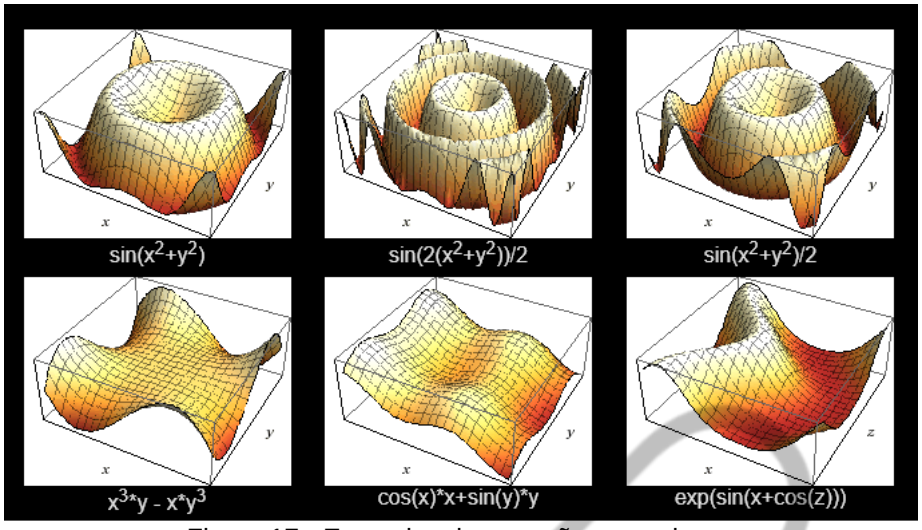


Figura 17 - Exemplos de equações complexas
Fonte: FractArte (2024)

Nestes casos, como podemos obter o vetor gradiente? Por meio de **derivadas**! A derivada de uma função é uma medida de quão rápido o valor da função muda quando a variável independente muda. Em termos práticos, nos diz a inclinação da função em qualquer ponto, **indicando se a função está aumentando, diminuindo ou permanecendo constante naquele ponto**.

Mas o nosso objetivo é **reduzir** o erro. Para minimizar a função de erro, queremos seguir na direção oposta ao gradiente. Isso é conhecido como **gradiente descendente**.

Imagine uma montanha com uma bola rolando nela. O vetor gradiente descendente é como um guia que diz à bola para rolar para baixo, em direção ao ponto mais baixo da montanha, que representa o mínimo global da função de erro.

Até o momento, imagino que você tenha entendido os artifícios matemáticos que otimizam uma função, e os usamos para otimizar a função de custo. **Mas como empregamos, de fato, o vetor gradiente descendente calculado para atualizar os pesos e bias?**

Para responder esta questão, considere uma rede simples com 2 pesos (p1, p2) e 1 bias (b). Imagine que numa determinada iteração do treinamento o vetor gradiente descendente foi [0,5; 50; -2.3], respectivamente, para [p1; p2; b]. **É importante destacar que o vetor gradiente descendente não revela o quanto atualizar cada parâmetro, mas sua direção e intensidade**. Da seguinte forma:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Peso 1 (0,5):** o gradiente descendente é positivo, indicando que aumentar o peso 1 diminuiria o custo.
- **Peso 2 (50):** o gradiente é muito maior em magnitude comparado ao peso 1, isso sugere que o peso 2 tem um impacto muito maior no custo. Novamente, porque o gradiente descendente é positivo, queremos aumentar esse peso para diminuir o custo.
- **Bias (-2,3):** o gradiente descendente é negativo, o que implica que diminuir o bias reduziria o custo. Portanto, para minimizar o custo, devemos diminuir o valor corrente do bias.

Na prática, faríamos essas atualizações multiplicando cada componente do gradiente descendente pela taxa de aprendizado e então somando o resultado dos valores atuais dos parâmetros. Se **α** é a taxa de aprendizado, a regra de atualização para cada parâmetro **θ** seria:

$$\theta atualizado = \theta corrente + \alpha * (gradiente\ descendente\ correspondente)$$

Agora precisamos relembrar um ponto crucial: **queremos minimizar o erro médio**, logo, devemos passar por todos os exemplos do conjunto de treino. Mas, e se o conjunto de treino for muito grande? É daí que surgiu o conceito de **vetor gradiente descendente estocástico**.

No cálculo do vetor gradiente descendente estocástico não usamos todos os dados do treino de uma vez para atualizar os pesos. Nós separamos aleatoriamente o treino em subconjuntos denominados **batches** (lotes). Calculamos o vetor gradiente descente médio apenas para um batch. Na próxima atualização, usamos outro batch e assim sucessivamente. **Após utilizarmos todos os batches disponíveis, dizemos que completamos uma época do treinamento**. Daí voltamos a usar o primeiro batch, depois o segundo, e assim por diante, por quantas épocas forem necessárias para convergir o treinamento.

Importante: uma época é um ciclo completo de treinamento em todo o conjunto de dados de TREINO!

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Para finalizar, vale ressaltar um detalhe: o processo de otimização da função de custos por meio do Backpropagation é um processo complexo e rico em detalhes matemáticos. Aqui passamos a intuição de funcionamento desta técnica. A seguir, iremos ver como tais estratégias de otimização são escolhidas na montagem da arquitetura da rede e seus impactos nos resultados.

3.4.3 Até quando treinar redes neurais multicamadas?

O ponto ideal de parada do treinamento pode variar dependendo do problema, do tamanho do conjunto de dados e da arquitetura da rede, mas existem algumas estratégias gerais para determinar quando parar.

À medida que o treinamento avança, o desempenho do modelo na validação é monitorado. Quando o desempenho na validação para de melhorar ou começa a piorar, isso pode ser um sinal de que o modelo está começando a “supertreinar” (overfitting). Nesse ponto, interrompe-se o treinamento para evitar que a rede memorize os dados de treinamento e não generalize bem para novos dados. Observe a figura:

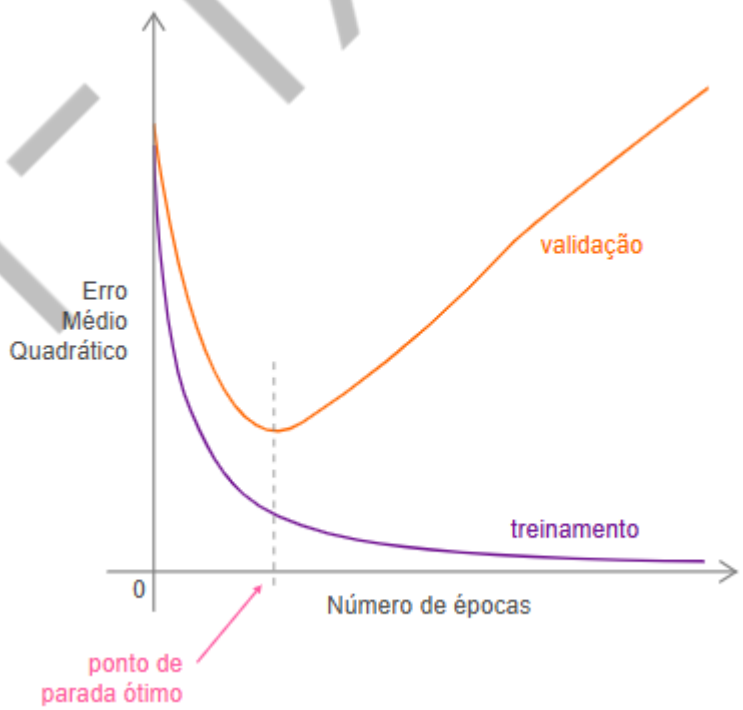


Figura 18 - Até quando treinar redes neurais multicamadas
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A figura “Até quando treinar redes neurais multicamadas” ilustra o ponto ótimo de parada do treinamento da rede neural: durante o treinamento, o erro associado ao conjunto de treinamento tende sempre a diminuir, assim como o erro associado ao conjunto de validação (que está atuando como um teste). A partir de um determinado ponto, a rede continuaria aprendendo diminuindo o erro do treinamento, mas passaria a aumentar o erro da validação, indicando a memorização do treino (overfitting). A parada no momento logo anterior a este ponto é a parada ideal.

Outra técnica é o **uso de regularização**, como a regularização L1 ou L2, que adiciona uma penalidade aos pesos da rede para evitar que se tornem muito grandes. Isso ajuda a controlar o overfitting e permite que o treinamento seja estendido por mais épocas sem comprometer o desempenho.

Além disso, é importante estar ciente dos recursos computacionais disponíveis. Treinar redes neurais multicamadas pode ser intensivo em termos de tempo e recursos. Portanto, em alguns casos, pode ser necessário interromper o treinamento devido a restrições de tempo ou hardware. Isso é implementado **indicando o número máximo de épocas que a rede pode treinar**.

Concluindo, o momento ideal para interromper o treinamento de redes neurais multicamadas é quando o desempenho na validação não melhora mais ou começa a piorar, indicando overfitting. Empregar estratégias para evitar o overfitting (como acompanhar as métricas de validação e/ou usar técnicas de regularização) é essencial para determinar o momento de parada do treinamento.

3.5 Hiperparâmetros

Ao projetar a estrutura de uma rede neural multicamada, é necessário considerar diversos hiperparâmetros. Embora muitos deles possam ter seus valores inicialmente definidos com base em heurísticas (como a escolha do número de camadas ocultas e de neurônios em cada camada), todos eles, no final, dependem diretamente das características dos dados analisados.

Assim, a otimização de hiperparâmetros em redes neurais é tão crucial quanto em outros métodos de aprendizado de máquina. O principal desafio está na

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

quantidade de hiperparâmetros a serem ajustados e no tempo necessário para treinar cada combinação.

Para tornar mais clara a compreensão sobre os hiperparâmetros, listaremos os mais importantes, explicando seus significados, valores comumente utilizados e possíveis impactos nos resultados. Utilizaremos os nomes dos hiperparâmetros adotados pela biblioteca Scikit-learn como referência:

- **`hidden_layer_sizes`:** define a configuração da rede neural, especificando o número de camadas ocultas e a quantidade de neurônios em cada uma delas. Por exemplo, `hidden_layer_sizes=(100, 50)` representa uma rede com duas camadas ocultas, sendo que a primeira possui 100 neurônios e a segunda, 50.
- **`activation`:** determina a função de ativação aplicada nas camadas ocultas. As opções mais utilizadas incluem `relu` (Rectified Linear Unit), `logistic` (função sigmoide) e `tanh` (tangente hiperbólica).
- **`solver`:** indica o algoritmo de otimização utilizado no treinamento. Entre os métodos disponíveis estão `adam` (otimizador Adam), `sgd` (Gradiente Descendente Estocástico) e `lbfgs` (um método Quasi-Newton, mais apropriado para pequenos conjuntos de dados).
- **`alpha`:** ajusta a intensidade da regularização L2 durante o treinamento. Valores maiores de `alpha` implicam em regularização mais forte.
- **`learning_rate`:** define a estratégia de atualização da taxa de aprendizado no processo de otimização.
- **`learning_rate_init`:** determina a taxa de aprendizado inicial, válida apenas quando `learning_rate` é configurado como `constant`.
- **`max_iter`:** especifica o número máximo de iterações (ou épocas) permitidas durante o treinamento. Caso o algoritmo não tenha convergido até atingir esse limite, o treinamento será interrompido.
- **`batch_size`:** indica o número de amostras processadas em cada iteração do algoritmo de otimização.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **`shuffle`:** controla se os dados devem ser embaralhados em cada iteração do treinamento.
- **`random_state`:** define a semente do gerador de números aleatórios para garantir reprodutibilidade nos resultados.
- **`tol`:** estabelece a tolerância para determinar a convergência do algoritmo, representando a diferença mínima de desempenho entre iterações consecutivas.
- **`verbose`:** configura a exibição de mensagens de progresso durante o treinamento.
- **`early_stopping`:** habilita a interrupção antecipada do treinamento caso não haja melhora no desempenho em um conjunto de validação.

Esses são apenas alguns dos hiperparâmetros disponíveis no Scikit-learn para o treinamento de redes neurais multicamadas. Para uma compreensão mais detalhada e uma lista completa, recomendamos consultar a documentação oficial da biblioteca.

É importante ressaltar que a escolha adequada desses hiperparâmetros pode impactar significativamente o desempenho e a capacidade de generalização da rede neural. Testar diferentes combinações de valores e avaliar o desempenho do modelo utilizando validação cruzada é um passo essencial no ajuste de hiperparâmetros para obter modelos mais robustos e eficientes.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 HANDS ON: MODELAGENS PREDITIVAS COM REDES NEURAIS

4.1 Sobre a base de dados

Vamos utilizar a base “Iris” presente no Scikit-learn. Esta base é um conjunto de dados muito conhecido na área de aprendizado de máquina e classificação. Ele contém informações sobre três espécies diferentes de flores Iris: setosa, versicolor e virginica. Cada amostra da base de dados inclui quatro atributos (características): comprimento e largura da sépala, e comprimento e largura da pétala, todos medidos em centímetros. O objetivo típico ao trabalhar com o conjunto de dados Iris é criar um modelo de aprendizado de máquina que possa classificar corretamente a espécie de uma flor Iris com base em suas características.

Para efeitos de curiosidade, a figura a seguir ilustra os tipos de flores deste dataset.

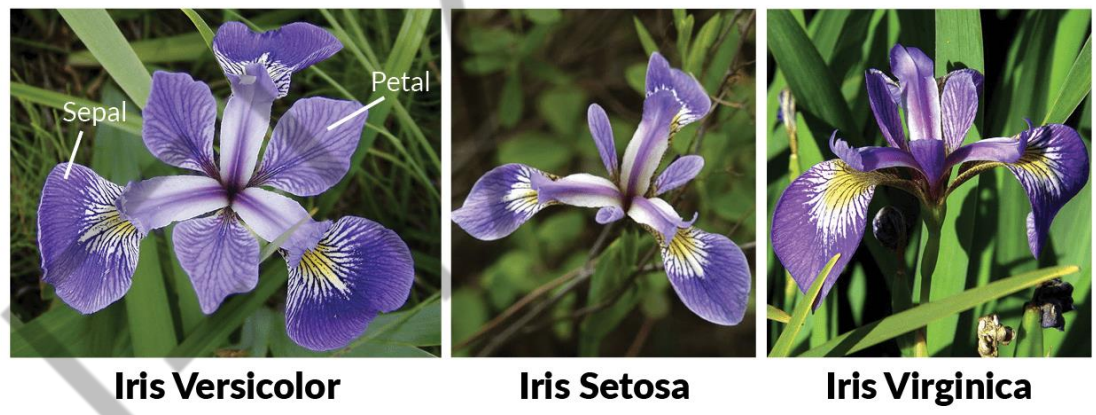


Figura 19 - Tipos de flores do “Iris Dataset”
Fonte: Lac.inpe (2024)

Atenção: para esta parte do código, utilizaremos um dataset já salvo na memória do Scikit-learn, por isso não há um arquivo CSV associado!

4.2 Modelagem via Perceptron

Vamos começar com os **imports** e a **montagem da base de dados**:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Apenas para evitar warnings
import warnings
warnings.filterwarnings('ignore')

# Bibliotecas de Data Science
import pandas as pd
import seaborn as sns

# Separação dos dados
from sklearn.model_selection import train_test_split

# Modelos a serem utilizados
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier

# Métricas de avaliação
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.metrics import accuracy_score


# Lendo o montando o DataFrame com o dataset
from sklearn.datasets import load_iris
iris = load_iris()

# Transformando em Dataframe Pandas:
#   - os dados são iris.data
#   - as colunas são os nomes das características
data = pd.DataFrame( iris.data,
columns=iris.feature_names )
data["class"] = iris.target
data
```

Código-fonte 1 - Imports iniciais e leitura da base
Fonte: Elaborado pelo autor (2024)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	class
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

150 rows × 5 columns

Figura 20 - Resultado do código-fonte 1
Fonte: Elaborado pelo autor (2024)

Como se trata de um dataset fortemente empregado para ensino de Machine Learning, ele já se encontra muito bem formatado: não há valores faltantes, dados duplicados ou qualquer outro "problema grave" que precisa ser sanado. Todas as variáveis são expressas na mesma medida (centímetros) e seus valores são próximos, dispensando padronização ou normalização. Portanto, podemos pular as etapas de limpeza e exploração. Reforçamos que tais etapas são essenciais ao trabalharmos com datasets reais.

Podemos seguir diretamente na divisão dos dados em treino e teste, e aqui separaremos 20% dos dados para teste. Usaremos o "`random_state=42`" para efeitos de reprodutibilidade:

```
# Divisão em TREINO e TESTE
X = data.drop(columns=["class"]) # variáveis
y = data["class"]                # label

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Código-fonte 2 - Divisão dos dados em treino e teste
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O código-fonte “Divisão dos dados em treino e teste” não produz saídas. Então avancemos para a definição e treinamento do Perceptron, bem como sua avaliação pela matriz de confusão:

```
# Cria um perceptron e o treina (com o fit())
p = Perceptron( random_state=42 )
p.fit(X_train, y_train)

# Salva as predições do TESTE na variável y_pred
y_pred = p.predict(X_test)

# Matriz de confusão do perceptron
from sklearn.metrics import confusion_matrix

def matriz_confusao(teste_labels, teste_preds, labels):
    cm = confusion_matrix(teste_labels, teste_preds,
labels=labels)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                                  display_labels=labels)
    disp.plot(cmap="Blues")

matriz_confusao(y_test, y_pred, p.classes_)
```

Código-fonte 3 - Definição e treinamento do Perceptron
Fonte: Elaborado pelo autor (2024)

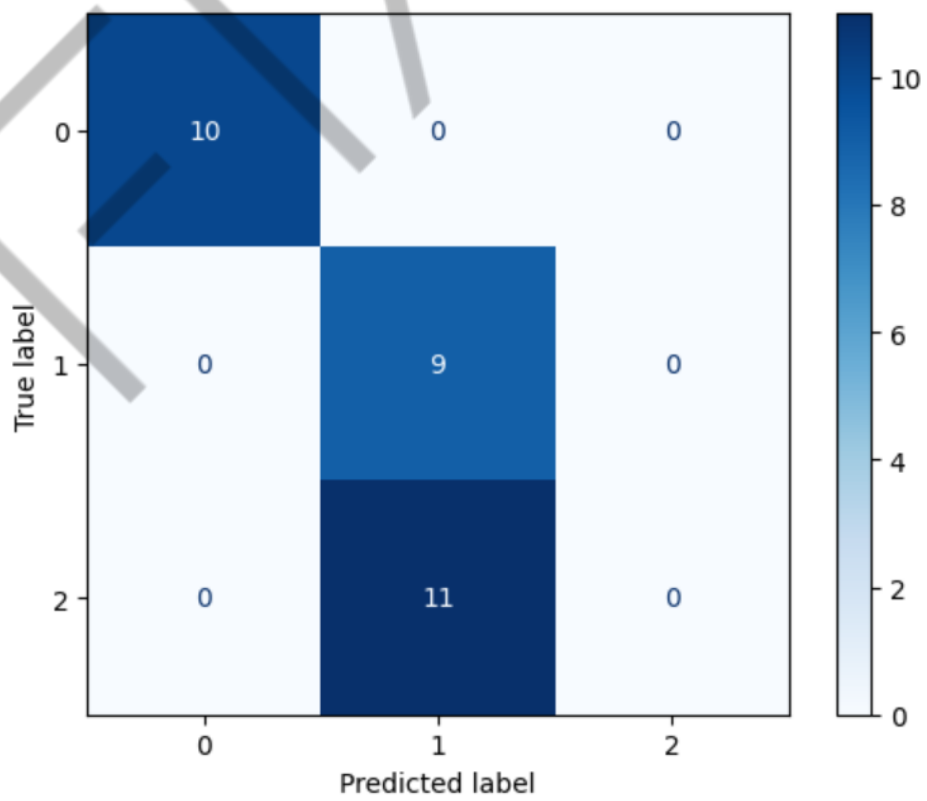


Figura 21 - Resultado do código-fonte 3
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O resultado não foi muito bom, não é? Vimos que o Perceptron resolve problemas linearmente separáveis, mas será que é o caso deste dataset? Vamos investigar diversas plotagens dos dados originais:

```
sns.pairplot( data=data, vars=('sepal length (cm)', \
                               'sepal width (cm)', \
                               'petal length (cm)', \
                               'petal width (cm)'), \
              hue='class' );
```

Código-fonte 4 - Plotando o Iris Dataset
Fonte: Elaborado pelo autor (2024)

O código-fonte “Plotando o Iris Dataset” produz uma visualização muito rica que nos permite investigar o comportamento do dataset. Observe a figura:

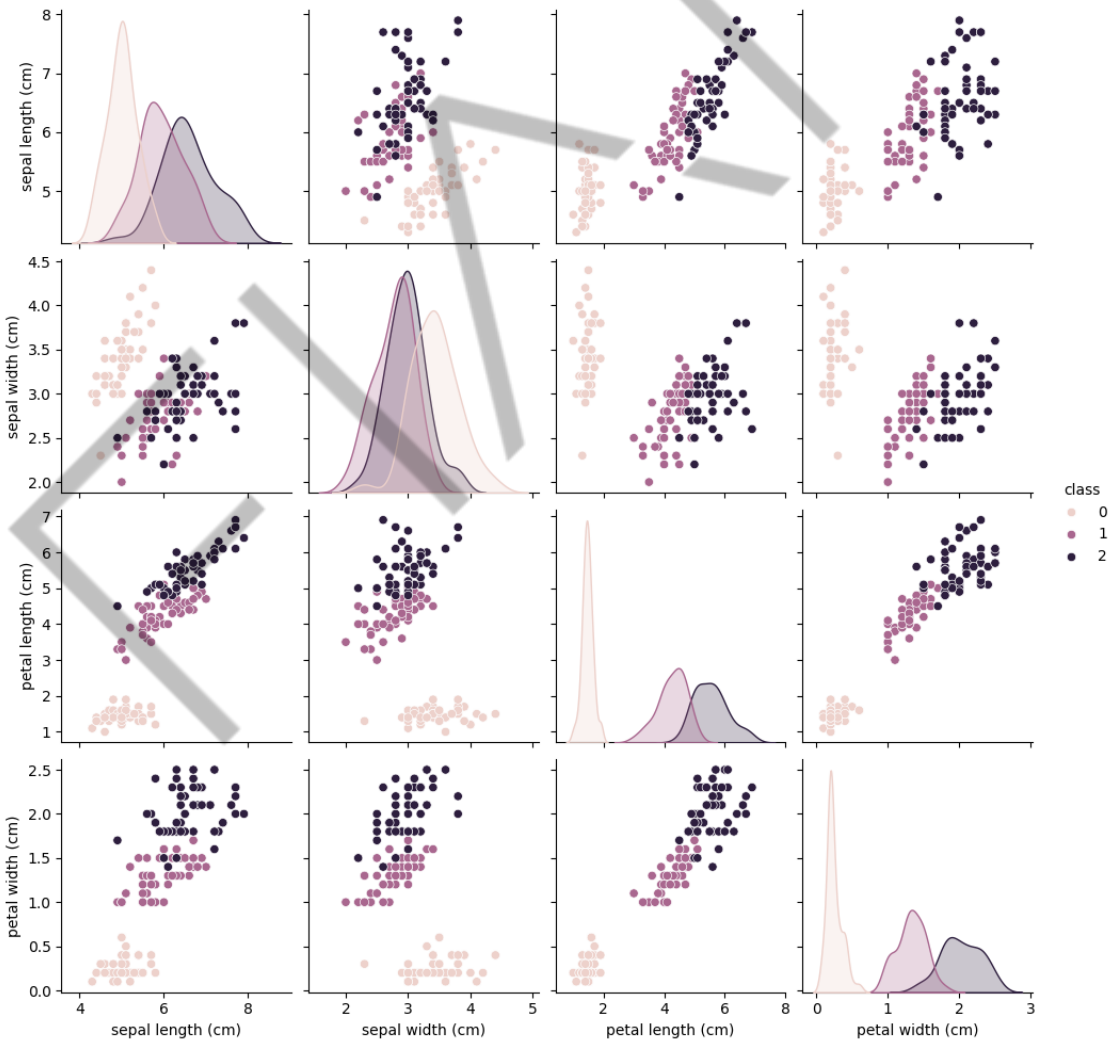


Figura 22 - Resultado do código-fonte 4
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A figura “Resultado do código-fonte 4” mostra, sob diversos aspectos, que existe uma clara separação dos pontos da classe “0” em relação às demais, porém as classes “1” e “2” podem ser confundidas. É exatamente esse comportamento que a matriz de confusão da figura “Resultado do código-fonte 3” apontou: as classes “1” e “2” são confundidas como uma única classe. Portanto, precisamos de um modelo mais sofisticado.

4.3 Modelagem via Redes Multicamadas

Nosso novo modelo será uma Rede Neural Multicamadas representada pela figura “Representação da nossa primeira Rede Neural Artificial Multicamadas”, contendo apenas uma camada oculta com cinco neurônios.

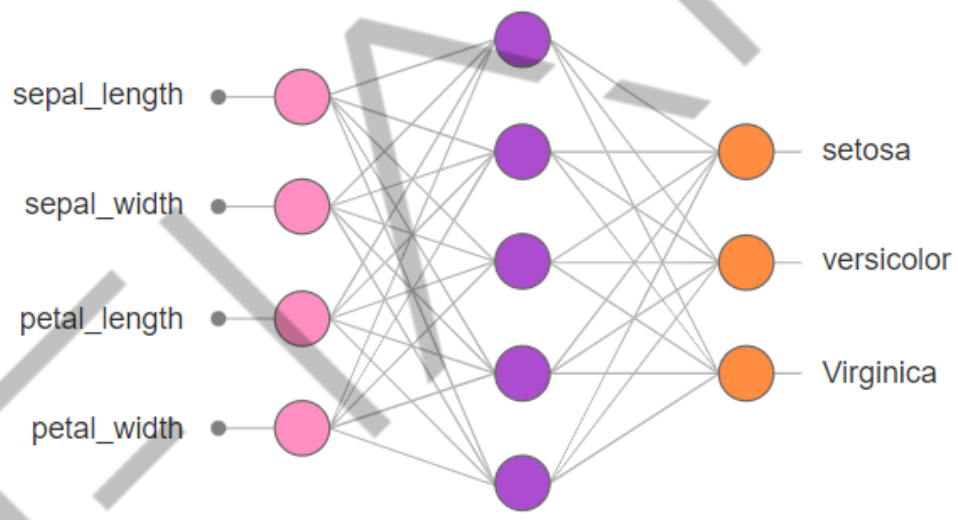


Figura 23 - Representação da nossa primeira Rede Neural Artificial Multicamadas
Fonte: Medium (2019), adaptado por FIAP (2025)

O código para sua definição, treinamento e avaliação está logo a seguir. Ele define a arquitetura como esperado (1 camada oculta com 5 neurônios), mostrará as saídas na tela e treinará por no máximo 200 épocas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
#Arquitetura
mlp = MLPClassifier(
    hidden_layer_sizes=(5,),
    verbose=True,
    max_iter=200,
    random_state=1
)

# Treinamento
mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

# Avaliação
matriz_confusao(y_test, y_pred, mlp.classes_)
```

Código-fonte 5 - Primeiro modelo com Rede Neural Multicamadas
Fonte: Elaborado pelo autor (2024)

A saída está parcialmente apresentada na figura a seguir. Sua saída deve conter 200 linhas de informação (devido ao hiperparâmetro "`verbose=True`"), e terminar com a matriz de confusão a seguir. Observe:

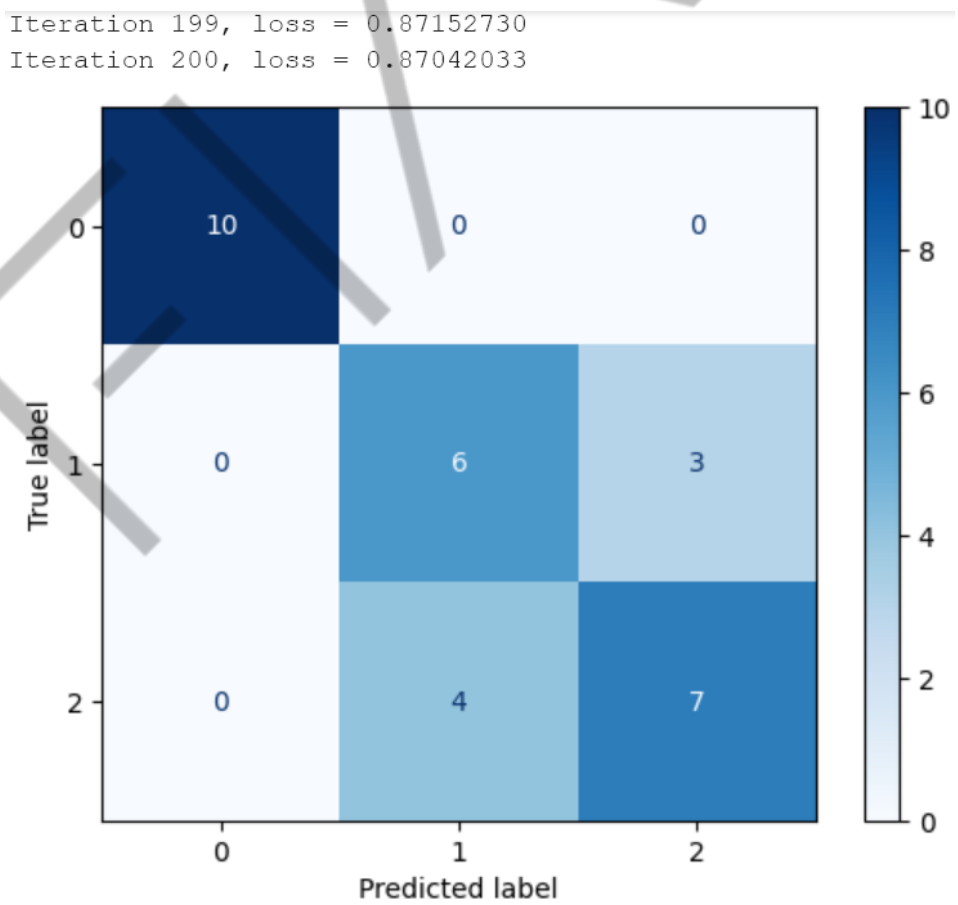


Figura 24 - Resultado do código-fonte 5
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Podemos observar que o modelo proposto não fez milagre. Por mais que os exemplos das classes “1” e “2” estejam começando a serem mais bem acomodados, ainda há uma mistura dos *labels*. Portanto, é necessário aprimorar nosso modelo alterando os hiperparâmetros da rede neural.

Vamos começar alterando a **função de ativação**.

```
#Alterando a função de ativação...
mlp = MLPClassifier(
    hidden_layer_sizes=(5,),
    verbose=False,
    max_iter=50,
    random_state=1,
    activation="logistic"
)

mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

matriz_confusao(y_test, y_pred, mlp.classes_)
```

Código-fonte 6 - Segundo modelo com Rede Neural Multicamadas
Fonte: Elaborado pelo autor (2024)

E vejamos o resultado:

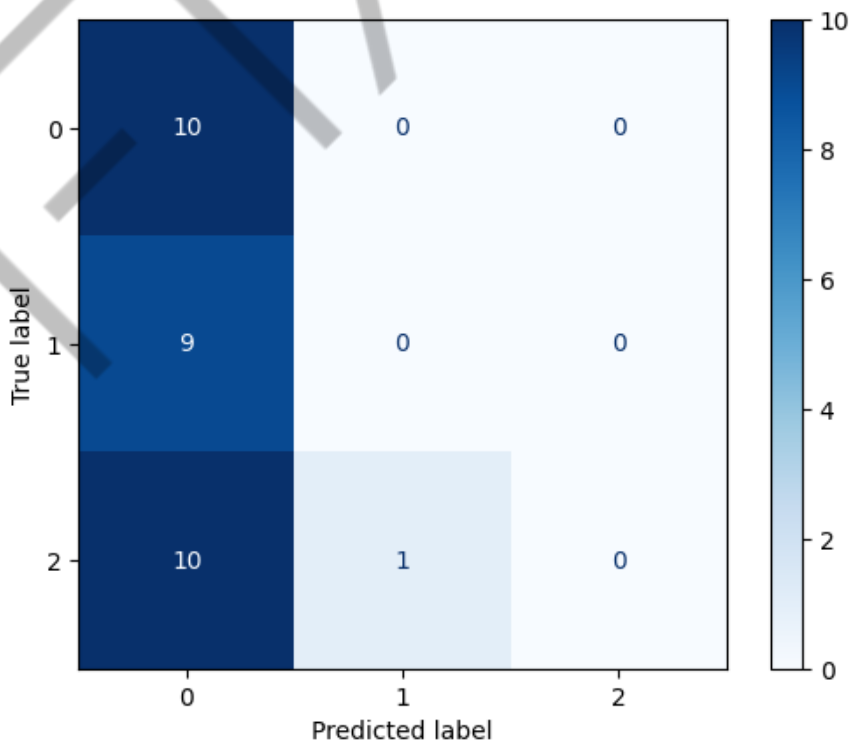


Figura 25 - Resultado do código-fonte 6
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Piorou! Será que precisamos alterar o “solver” para o algoritmo de otimização da rede mais recomendado para datasets pequenos? Esta é uma dica que a própria documentação do Scikit-learn indica. Faremos isso com o comando:

```
     #Alterando o solver para o que a própria documentação
aconselha para datasets menores
     mlp = MLPClassifier(
         hidden_layer_sizes=(5,),
         verbose=False,
         max_iter=50,
         random_state=1,
         activation="logistic",
         solver="lbfgs"
     )

     mlp.fit(X_train, y_train)
     y_pred = mlp.predict(X_test)

     print("ACC Teste: ", accuracy_score(y_test, y_pred))

     matriz_confusao(y_test, y_pred, mlp.classes_)
```

Código-fonte 7 - Terceiro modelo com Rede Neural Multicamadas
Fonte: Elaborado pelo autor (2024)

A saída está na figura a seguir. Aqui obtivemos 100% de acerto, e isso não é overfitting, pois tal métrica incrível está calculada no “teste”, ou seja, em outra base diferente do “treino”. Realmente, a rede aprendeu e soube generalizar seu conhecimento em novos dados. Observe:

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

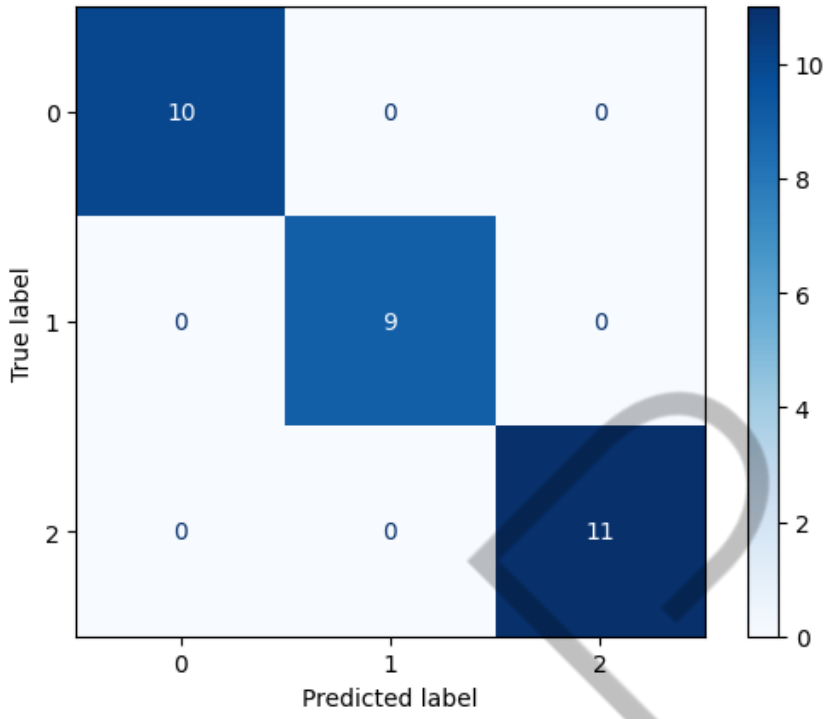


Figura 26 - Resultado do código-fonte 7
Fonte: Elaborado pelo autor (2024)

Agora, alterar o “solver” é a única saída? Não! Aprendemos diversos hiperparâmetros disponíveis, e certamente a combinação de outros também levaria a bons resultados. Por outro lado, outras combinações podem não ser tão boas. Vejamos o efeito de combinar outro solver, com uma arquitetura mais profunda (2 camadas ocultas), e outra política de atualização da taxa de aprendizagem:

```
#Testando outra arquitetura...
mlp = MLPClassifier(
    hidden_layer_sizes=(5,3),
    max_iter=1000,
    random_state=1,
    activation="logistic",
    learning_rate="adaptive",
    solver="adam"
)

mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

print("ACC Teste: ", accuracy_score(y_test, y_pred))

matriz_confusao(y_test, y_pred, mlp.classes_)
```

Código-fonte 8 - Quarto modelo com Rede Neural Multicamadas
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O resultado está na figura a seguir. Novamente, o resultado não foi muito bom. Observe:

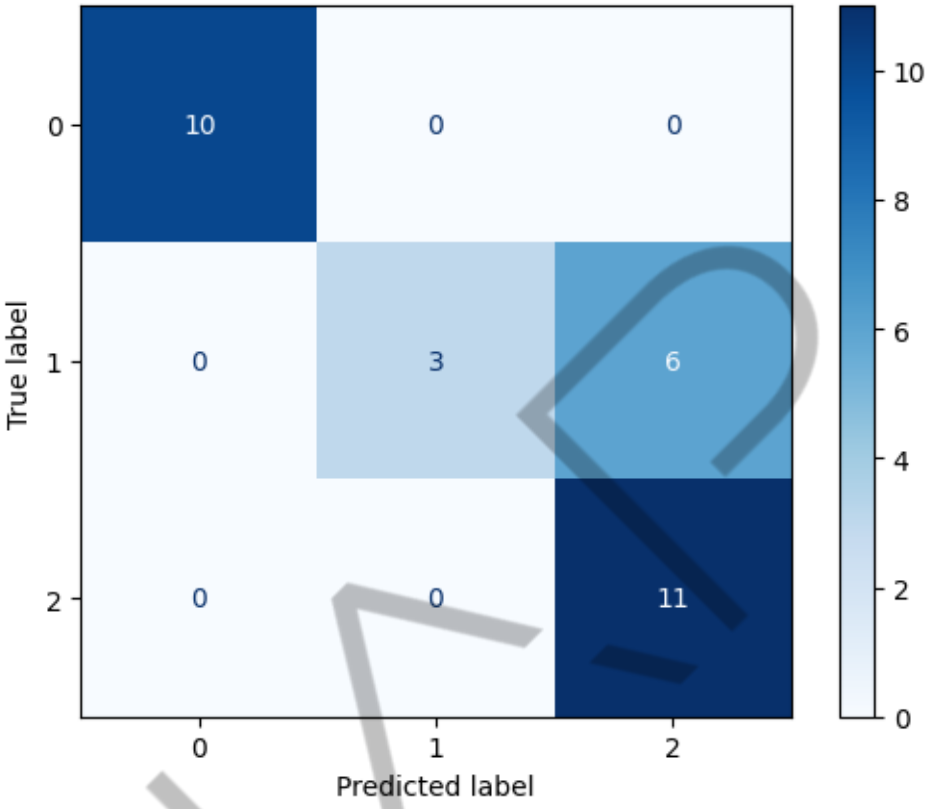


Figura 27 - Resultado do código-fonte 8
Fonte: Elaborado pelo autor (2024)

O que o último modelo nos mostra é que a quantidade de combinações de hiperparâmetros é muito grande, e que algumas delas levarão a bons resultados enquanto muitas outras não. Isso reforça a importância de realizar otimização de hiperparâmetros em validação cruzada também nos modelos de redes neurais multicamadas.

Conclusão: a parametrização de redes neurais multicamadas é uma tarefa desafiadora, pois envolve a escolha cuidadosa de hiperparâmetros que afetam diretamente o desempenho e a capacidade de generalização do modelo. Determinar o número ideal de camadas, neurônios, taxas de aprendizado, entre outros fatores, não é trivial, já que as melhores configurações dependem das características dos dados e do problema em questão.

Essa complexidade exige uma busca sistemática por combinações de hiperparâmetros, muitas vezes utilizando técnicas como *grid search*. Embora o

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

processo possa ser computacionalmente intensivo, ele é essencial para alcançar modelos mais robustos e eficientes.

4.4 Redes Multicamadas de Regressão

As RNAs também podem ser empregadas em problemas de regressão. No código-fonte “Leitura do dataset advertising.csv”, veremos uma modelagem clássica de predição de vendas baseado na alocação de recursos em diversos canais de propaganda. Esta é uma tarefa muito relevante em Marketing.

Dica: para esta parte do código, utilizaremos o arquivo “advertising.csv”!

A base de dados é composta pelas seguintes variáveis:

- **TV:** valor gasto em publicidade na televisão.
- **Radio:** valor gasto em publicidade no rádio.
- **Newspaper:** valor gasto em publicidade em jornais.
- **Sales:** número de vendas do produto ou serviço.

Começaremos pelos imports, definição da função de avaliação dos modelos e leitura da base de dados:

```
# Ignorar warnings
import warnings
warnings.filterwarnings("ignore")

# Bibliotecas para uso e visualização de dados
import numpy as np
import pandas as pd

# Imports para seleção de modelos
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV

# Imports para preparação de dados
from sklearn.preprocessing import StandardScaler

# Import de modelos preditivos
from sklearn.linear_model import LinearRegression
from sklearn.neural_network import MLPRegressor
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Import de métricas
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

def metricas(X_tr_scaled, y_tr, y_ts, y_pr, model, id_modelo):

    y_pr_tr = model.predict(X_tr_scaled)

    # Erro quadrático médio
        print(f"MSE do TREINO ({id_modelo}): ", mean_squared_error(y_tr, y_pr_tr))
        print(f"MSE do TESTE ({id_modelo}): ", mean_squared_error(y_ts, y_pr))

    # Erro absoluto médio
        print(f"MAE do TREINO ({id_modelo}): ", mean_absolute_error(y_tr, y_pr_tr))
        print(f"MAE do TESTE ({id_modelo}): ", mean_absolute_error(y_ts, y_pr))

    # R²
    print(f"R² do TREINO ({id_modelo}): ", r2_score(y_tr, y_pr_tr))
    print(f"R² do TESTE ({id_modelo}): ", r2_score(y_ts, y_pr))
```

Código-fonte 9 - Leitura do dataset advertising.csv
Fonte: Elaborado pelo autor (2024)

Observe a figura:

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9

Figura 28 - Saída da execução do código-fonte 9
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Como os dados desta base estão bem-comportados, já podemos partir para a separação de *features* e *label*, bem como dividir os exemplos entre TREINO e TESTE. Uma vez dividida a base nestes dois conjuntos disjuntos, aplicamos a padronização. A próxima célula não gera saída.

```
# Separação de DADOS e LABEL
X = df.drop(columns=["Sales"])
y = df["Sales"]

# Dividindo dados para TREINO e TESTE
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

# Padronização dos dados
ss = StandardScaler()
X_train_scaled = ss.fit_transform(X_train)
X_test_scaled = ss.transform(X_test)
```

Código-fonte 10 - Separação de dados da base “advertising.csv”
Fonte: Elaborado pelo autor (2024)

Uma vez com dados padronizados, vamos construir uma primeira versão de modelo preditivo. Novamente, empregaremos a regressão linear, já avaliando o modelo resultante via a função “*metricas()*” que construímos anteriormente.

```
# Treinando o modelo
modelo = LinearRegression()
modelo.fit(X_train_scaled, y_train)

# Fazendo as predições
y_pred = modelo.predict(X_test_scaled)

# Metricas para regressao linear
metricas(X_train_scaled, y_train, y_test, y_pred, modelo,
"Reg Linear")
```

Código-fonte 11 - Gerando o modelo baseline (regressão linear)
Fonte: Elaborado pelo autor (2024)

Observe a figura:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
MSE do TREINO (Reg Linear):  2.8392326607544245
MSE do TESTE  (Reg Linear):  2.54162403622915
MAE do TREINO (Reg Linear):  1.2836301223784408
MAE do TESTE  (Reg Linear):  1.1594875061090584
R² do TREINO (Reg Linear):  0.8919647339302077
R² do TESTE  (Reg Linear):  0.9091484341849798
```

Figura 29 - Saída da execução do código-fonte 11
Fonte: Elaborado pelo autor (2024)

As métricas da figura “Saída da execução do código-fonte 11” estão boas! Isso indica que os dados possuem uma relação próxima à linear. Mas queremos saber se podemos ir além!

4.4.1 Primeiro modelo com Rede Neural Artificial

Agora que temos um modelo baseline, verificaremos se a RNA apresenta uma solução melhor, ao permitir explorar relações não lineares nos dados. Para isso, construiremos uma RNA de regressão, utilizando todos os valores default dos hiperparâmetros (consulte-os na documentação da classe “MLPRegressor”). A única alteração será no número máximo de épocas, que será limitado em 1000.

```
# Treinando uma RNA padrão e realizando predições
modelo_rn = MLPRegressor(max_iter=1000, random_state=97)
modelo_rn.fit(X_train_scaled, y_train)
y_pred_rn = modelo_rn.predict(X_test_scaled)

# Metricas para RNA otimizada
metricas(X_train_scaled,  y_train,  y_test,  y_pred_rn,
modelo_rn, "RN")
```

Código-fonte 12 - Gerando o modelo RNA preliminar
Fonte: Elaborado pelo autor (2024)

Observe a figura:

```
MSE do TREINO (RN):  3.3393236244781193
MSE do TESTE  (RN):  5.636869802488712
MAE do TREINO (RN):  1.381398811609204
MAE do TESTE  (RN):  1.6621776019409256
R² do TREINO (RN):  0.8729358388798696
R² do TESTE  (RN):  0.7985073950546592
```

Figura 30 - Saída da execução do código-fonte 12
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.4.2 Aprimorando o modelo de Rede Neural Artificial

Novamente, o primeiro modelo baseado em RNA não saiu melhor que o baseline, portanto, temos que investigar as configurações de hiperparâmetros que fazem mais sentido aos dados apresentados.

Por exemplo, podemos tunar a rede aprimorando sua arquitetura, alterando o otimizador e a política de adaptação da taxa de aprendizagem.

```
     # Treinando uma RNA padrão e realizando predições
     modelo_otimizado  =  MLPRegressor(hidden_layer_sizes=(10,
5,),
                              max_iter=1000,
                              solver="lbfgs",
                              learning_rate="adaptive",
                              random_state=97)
     modelo_otimizado.fit(X_train_scaled, y_train)
     y_pred_rn = modelo_otimizado.predict(X_test_scaled)

     # Metricas para RNA otimizada
     metricas(X_train_scaled,  y_train,  y_test,  y_pred_rn,
modelo_otimizado, "RN OTIMIZADO")
```

Código-fonte 13 - Gerando o modelo RNA secundário – Seed 97
Fonte: Elaborado pelo autor (2024)

A figura “Saída da execução do código-fonte 13” mostra que o resultado melhorou, reforçando o conceito que os hiperparâmetros das redes neurais multicamadas devem ser investigados para melhor configuração do modelo.

```
MSE do TREINO (RN OTIMIZADO):  0.4379354351237677
MSE do TESTE  (RN OTIMIZADO):  1.9127624091307802
MAE do TREINO (RN OTIMIZADO):  0.4511983909532657
MAE do TESTE  (RN OTIMIZADO):  0.982569649097093
R² do TREINO (RN OTIMIZADO):  0.9833361767392409
R² do TESTE  (RN OTIMIZADO):  0.9316273935780587
```

Figura 31 - Saída da execução do código-fonte 13
Fonte: Elaborado pelo autor (2024)

Por outro lado, o que realmente fez diferença no código acima foi a semente aleatória utilizada “`random_state=97`”. Se nós rodássemos o mesmo código, porém alterando o valor da semente, por exemplo, para “`random_state=93`”, teríamos outro resultado:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    # Treinando uma RNA padrão e realizando predições
    modelo_otimizado  =  MLPRegressor(hidden_layer_sizes=(10,
5,),
                                max_iter=1000,
                                solver="lbfgs",
                                learning_rate="adaptive",
                                random_state=93)
    modelo_otimizado.fit(X_train_scaled, y_train)
    y_pred_rn = modelo_otimizado.predict(X_test_scaled)

    # Metricas para RNA otimizada
    metricas(X_train_scaled,  y_train,  y_test,  y_pred_rn,
modelo_otimizado, "RN OTIMIZADO")
```

Código-fonte 14 - Gerando o modelo RNA secundário – Seed 93
Fonte: Elaborado pelo autor (2024)

Observe a figura:

```
MSE do TREINO (RN OTIMIZADO):  0.7223982401992917
MSE do TESTE  (RN OTIMIZADO):  4.93884563479431
MAE do TREINO (RN OTIMIZADO):  0.5615115916845373
MAE do TESTE  (RN OTIMIZADO):  1.6007727270138643
R² do TREINO (RN OTIMIZADO):  0.9725121202052027
R² do TESTE  (RN OTIMIZADO):  0.8234586025140637
```

Figura 32 - Saída da execução do código-fonte 14
Fonte: Elaborado pelo autor (2024)

Agora executando novamente com "`random_state=3`":

```
    # Treinando uma RNA padrão e realizando predições
    modelo_otimizado  =  MLPRegressor(hidden_layer_sizes=(10,
5,),
                                max_iter=1000,
                                solver="lbfgs",
                                learning_rate="adaptive",
                                random_state=3)
    modelo_otimizado.fit(X_train_scaled, y_train)
    y_pred_rn = modelo_otimizado.predict(X_test_scaled)

    # Metricas para RNA otimizada
    metricas(X_train_scaled,  y_train,  y_test,  y_pred_rn,
modelo_otimizado, "RN OTIMIZADO")
```

Código-fonte 15 - Gerando o modelo RNA secundário – Seed 3
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
MSE do TREINO (RN OTIMIZADO):  0.7093816540612269
MSE do TESTE  (RN OTIMIZADO):  4.145685663426665
MAE do TREINO (RN OTIMIZADO):  0.6242040023304216
MAE do TESTE  (RN OTIMIZADO):  1.5046654466233291
R² do TREINO (RN OTIMIZADO):  0.9730074126009912
R² do TESTE  (RN OTIMIZADO):  0.8518104847410896
```

Figura 33 - Saída da execução do código-fonte 15
Fonte: Elaborado pelo autor (2024)

As figuras “Saída da execução do código-fonte 14” e “Saída da execução do código-fonte 15” mostram uma piora do modelo pelo simples fato de alterarmos a semente de inicialização aleatória utilizada no treinamento. Esta semente influencia, entre outras coisas, a inicialização dos pesos da rede. Dependendo da inicialização, pode ser mais “fácil” ou mais “difícil” treinar a rede, atingindo ótimos locais distintos.

Conclusão: como podemos observar no modelo de classificação, as redes de regressão também sofrem de resultados flutuantes de acordo com a arquitetura, a parametrização e as inicializações. Assim, a melhor forma de encontrar o modelo mais adequado é realizando otimização de hiperparâmetros.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

ACTIVATION function. **Wikipédia**, 22 de novembro de 2024. Disponível em: <https://en.wikipedia.org/wiki/Activation_function>. Acesso em: 03 dez. 2024.

ANATOMIA do Cérebro. **Portal do Médico**, 06 de julho de 2021. Disponível em: <https://blog.portaldomedico.com/anatomia-do-cerebro/>. Acesso em: 03 dez. 2024.

BATISTA, Bruno. Machine Learning sem código. **Medium**, 09 de julho de 2019. Disponível em: <https://medium.com/ensina-ai/machine-learning-sem-c%C3%B3digo-636d1a8f9081>. Acesso em: 03 dez. 2024.

CAPÍTULO 4 – O Neurônio, Biológico e Matemático. **Deep Learning Book**, 2024. Disponível em: <https://www.deeplearningbook.com.br/o-neuronio-biologico-e-matematico/>. Acesso em: 03 dez. 2024.

CASTRO, Thiago da Silva. Função de ativação degrau, função sigmóide (HAYKIN *et al*., 2009). **Researchgate**, julho de 2018. Disponível: <https://www.researchgate.net/figure/Figura-10-a-Funcao-de-ativacao-degrau-b-funcao-sigmoide-HAYKIN-et-al-2009_fig9_328938085>. Acesso em: 03 dez. 2024.

FERRÃO, Reginaldo. Modelo de Neurônio em 1943. **Medium**, 23 de abril de 2024. Disponível em: <https://medium.com/@regis1981/warren-macculloch-e-o-primeiro-modelo-de-neur%C3%B4nio-em-1943-2d1bfcdbc8be>. Acesso em: 03 dez. 2024.

FONSECA, Elton. Operadores lógicos. **Treinaweb**, fevereiro de 2020. Disponível em: <https://www.treinaweb.com.br/blog/operadores-logicos>. Acesso em: 03 dez. 2024.

FRACTARTE. Gráficos de equações matemáticas em 3D. **FractArte**, 2024. Disponível em: <https://www.fractarte.com.br/wolframalpha/3d-equations.php>. Acesso em: 04 dez. 2024.

GRÁFICO da função do 2º grau. **Dever de Casa Web Blogspot**, maio de 2017. Disponível em: <https://deverdecasa-web.blogspot.com/2017/05/grafico-da-funcao-do-2-grau.html>. Acesso em: 03 dez. 2024.

HARKIRAN78. Artificial Neural Networks and its Applications. **GeeksforGeeks**, 07 de agosto de 2024. Disponível em: <https://www.geeksforgeeks.org/artificial-neural-networks-and-its-applications/>. Acesso em: 03 dez. 2024.

HAYKIN, Simon. **Redes neurais**: princípios e prática. Bookman Editora, 2008.

HEATON, Jeff. **Introduction to neural networks with Java**. Heaton Research, Inc., 2008.

REGUNATH, Gavita. Understanding The Difference Between AI, ML, And DL: Using An Incredibly Simple Example. **Advancing Analytic**, 15 de dezembro de 2021. Disponível em: <https://www.advancinganalytics.co.uk/blog/2021/12/15/understanding-the-

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

difference-between-ai-ml-and-dl-using-an-incredibly-simple-example>. Acesso em: 03 dez. 2024.

RUSSELL, Stuart J.; NORVIG, Peter. **Inteligencia Artificial**: un enfoque moderno. 2004.

RUSSELL, Stuart J.; NORVIG, Peter. **Inteligencia Artificial**: un enfoque moderno. 2004.

SANTOS, Rafael. Data Science Example - Iris dataset. **Lac.inpe**, 2024. Disponível em: <http://www.lac.inpe.br/~rafael.santos/Docs/CAP394/WholeStory-Iris.html>. Acesso em: 03 dez. 2024.

SILVA, Ivan Nunes da; SPATTI, Danilo Hernane; FLAUZINO, Rogério Andrade. **Redes neurais artificiais para engenharia e ciências aplicadas**. 2010.

VON ZUBEN, Fernando J.; DE CASTRO, Leandro N. **Redes neurais**. Notas de aula, FEEC, Universidade Estadual de Campinas, 2003.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

Termo	Explicação
IA	Inteligência Artificial.
PLN	Processamento de Linguagem Natural.
RNA	Redes Neurais Artificiais.
CNNs	Redes Neurais Convolucionais.
RNNs	Redes Neurais Recorrentes.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com